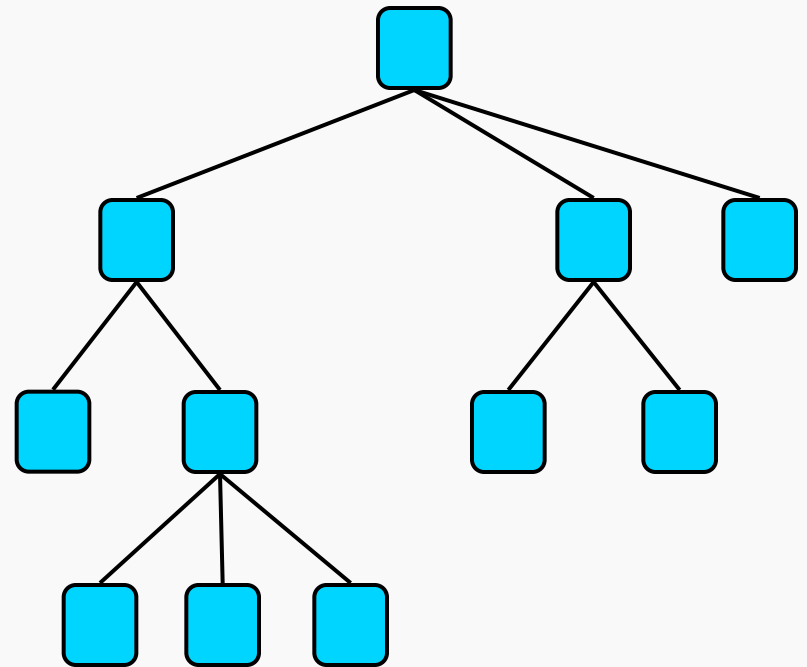
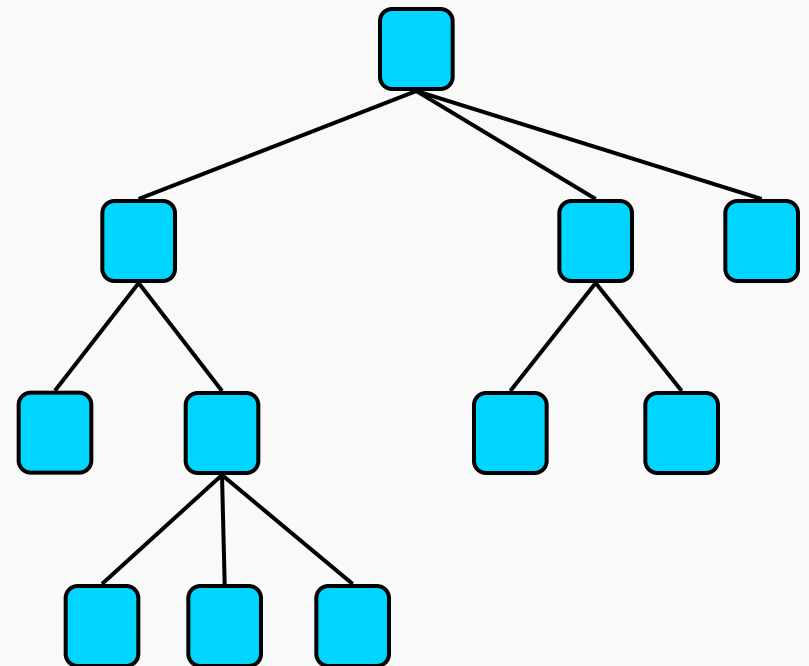

Computations in Trees

Saturation
Minimum Finding
Eccentricity
Center
Ranking



Trees

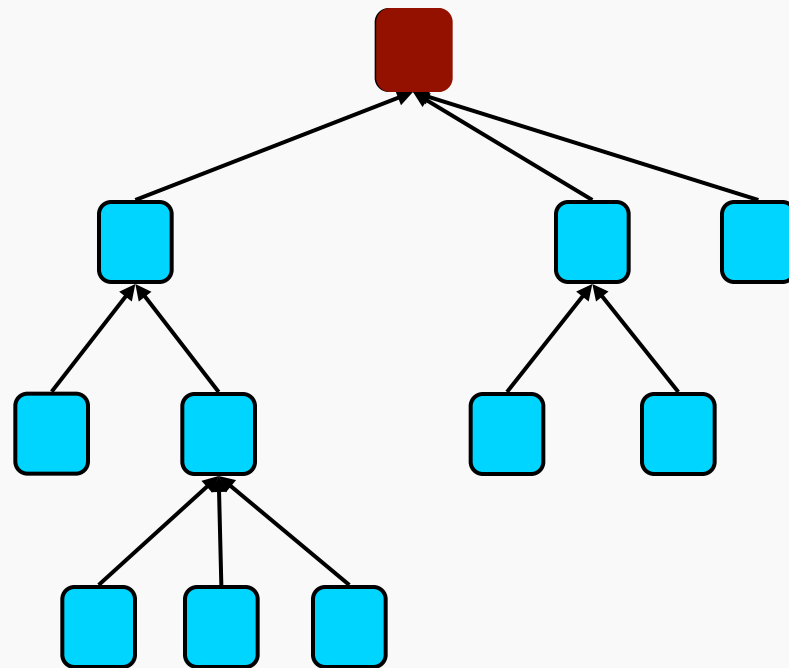
- Acyclic graph
- n entities
- $n - 1$ links



Rooted Trees

- Acyclic graph
- n entities
- $n - 1$ links

Rooted



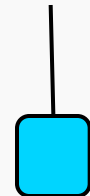
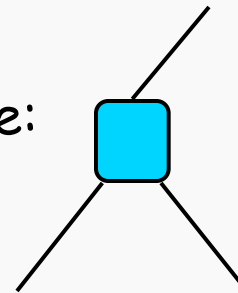
Saturation Technique

- Bidirectional links
- Ordered messages
- Full Reliability
- Knowledge of the topology

Note:

Each entity knows whether
it is a leaf:

or an internal node:



SATURATION: A Basic Technique

STATES = {available, awake, processing }

At the beginning, all entities are **available**

Arbitrary entities can start the computation (**multiple initiators**).

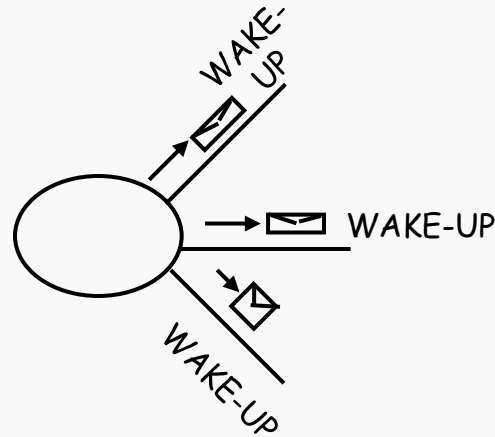
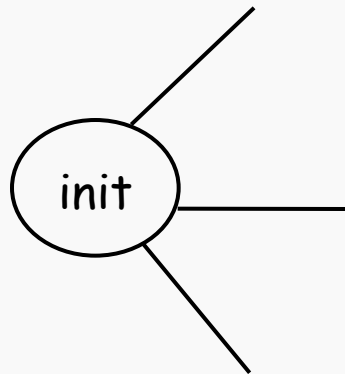
GOALS.

- ALL nodes must be **awaked**
- Two adjacent nodes must be **selected** (link election)
- The **two selected** nodes are ready to start any computation

SATURATION: A General Technique

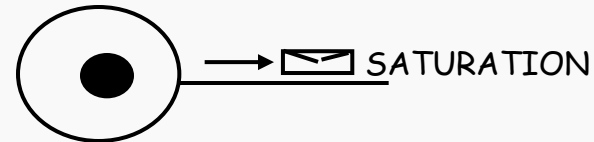
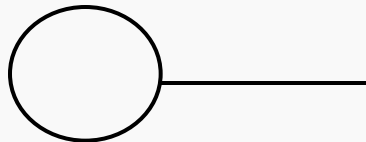
- **Activation phase:**
started by the initiators: all nodes are activated *wake-up*
- **Saturation Phase:**
started by all **leaves**: a unique pair of neighbours is identified
(saturated nodes)
- **Resolution Phase:**
a generic computation started by the **saturated nodes**

1)

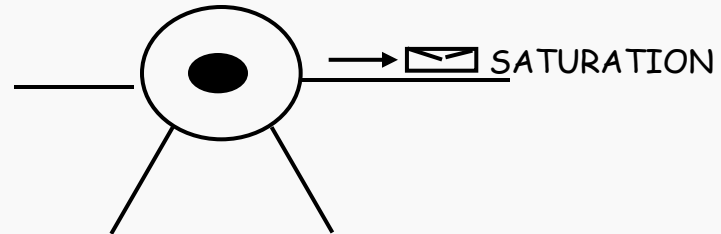
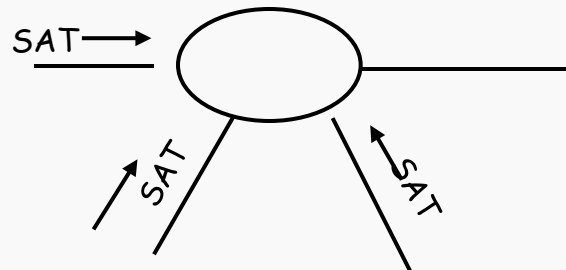


2)

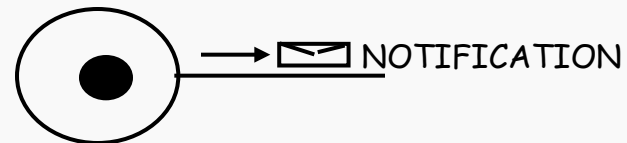
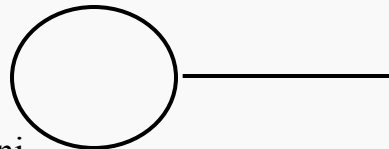
leaf



internal



3)



Paola Flocchini
Saturated node

$S = \{AVAILABLE, ACTIVE, PROCESSING, SATURATED\}$
 $S_{init} = AVAILABLE$

AVAILABLE I haven't been activated yet

```
Spontaneously (INITIATORS)
  send(Activate) to N(x);
  Neighbours := N(x)
  if |Neighbours|=1 then
    M := ("Saturation");           /* special case if
    parent ← Neighbours;          I am a leaf */
    send(M) to parent;
    become PROCESSING (saturation);
  else
    become ACTIVE;
```


Receiving(Activate)

send(Activate) to N(x)- {sender};

Neighbours:= N(x);

if |Neighbours|=1 then

M:=("Saturation");

parent $\leftarrow$ Neighbours; ** I am a leaf

send(M) to parent;

become PROCESSING; ** Saturation Phase

else

become ACTIVE;

I haven't started the saturation
phase yet

ACTIVE

Receiving(M)

Neighbours := Neighbours - {sender};

if |Neighbours| = 1 then

M := ("Saturation");

****Saturation Phase**

parent $\leftarrow$ Neighbours;

send(M) to parent;

become PROCESSING;

PROCESSING

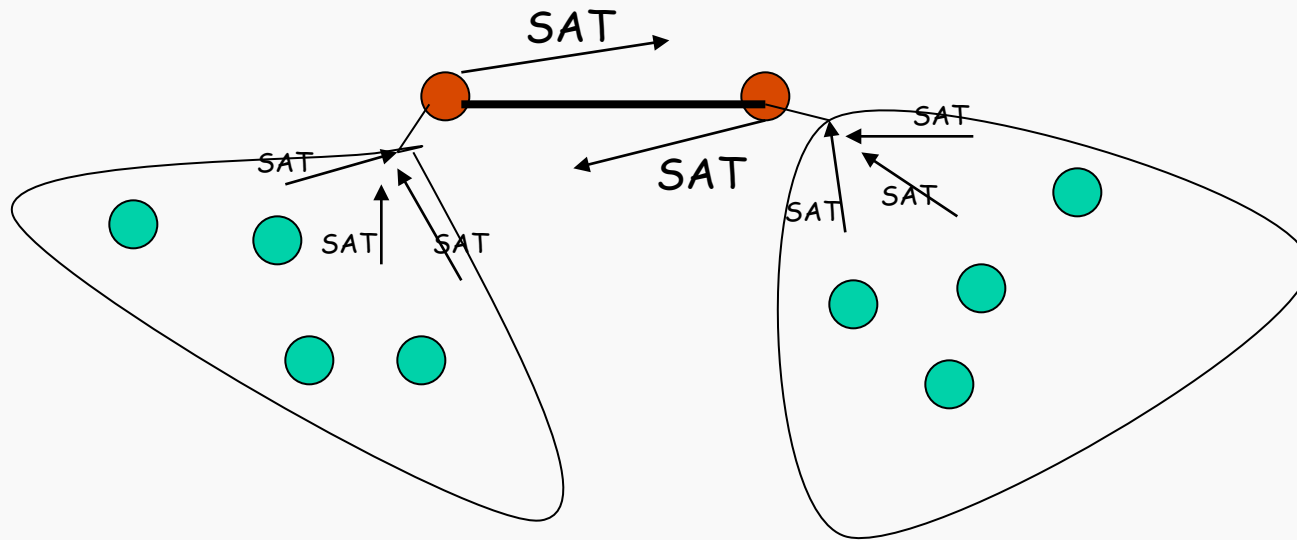
I have already started the
saturation phase and I am
receiving M from my Parent

receiving(M)

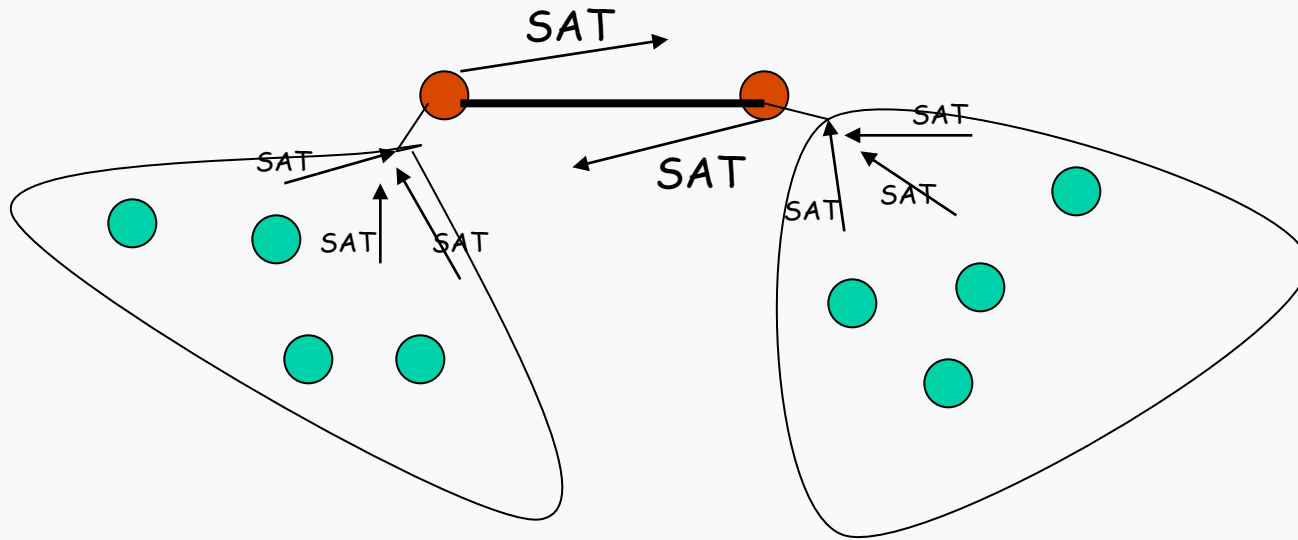
become SATURATED;

THEOREM:

Exactly two processing nodes become saturated, and they are neighbours.

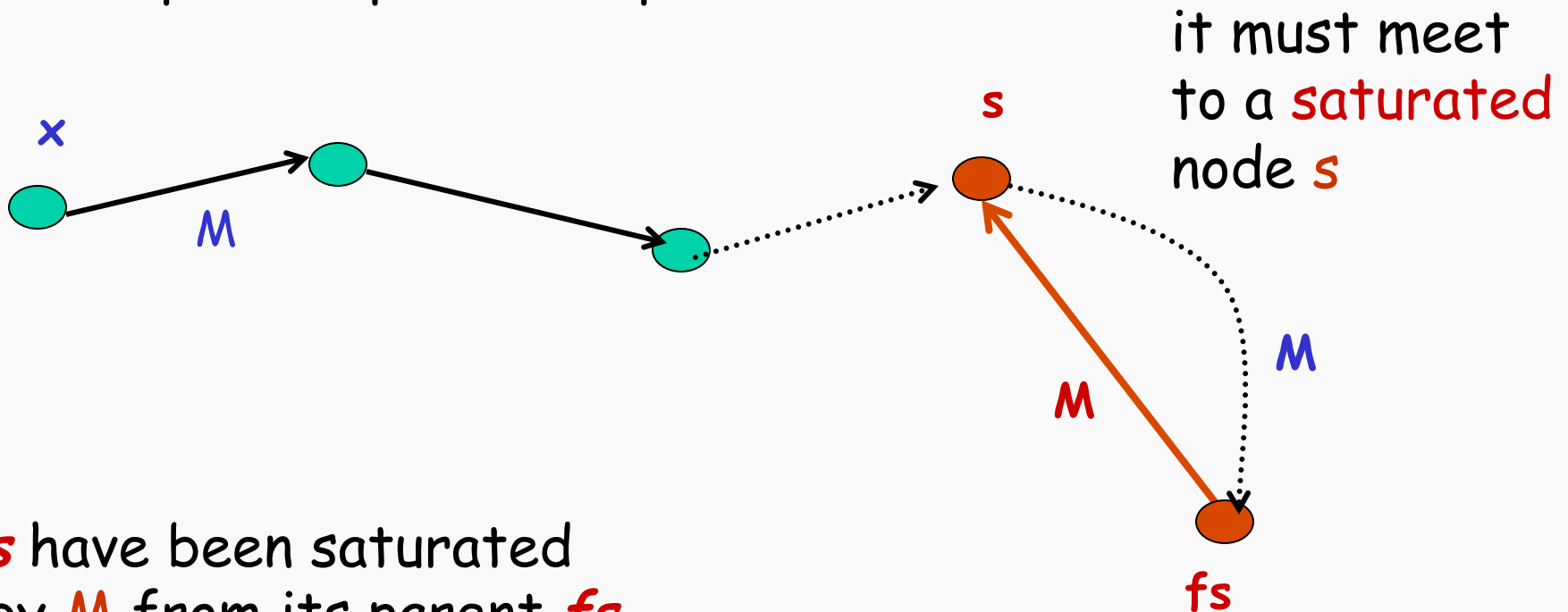


PROOF. A node becomes PROCESSING only after sending **saturation** to its **parent**.



A node become **SATURATED** only after receiving a message in the state PROCESSING from **its parent**

Choose a node x and consider the M -path back to its parent, parent of parent, ...

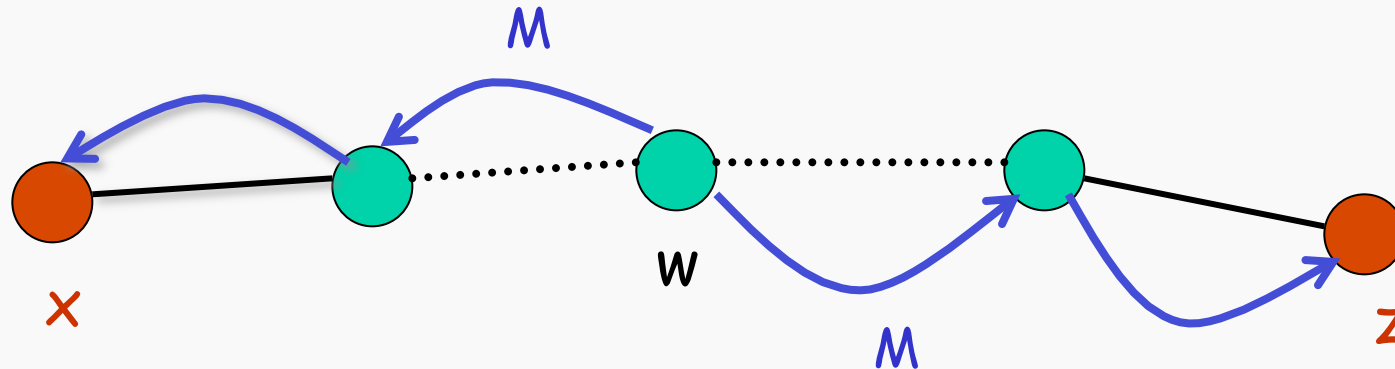


s have been saturated by M from its parent fs .

when receiving M , fs considered s as its parent as well --> fs gets saturated as well.

Why exactly two (adjacent) nodes ?

Assume there were **3**: **x, y, z**. Then, in a TREE, 2 of them must not be **linked**. So, consider the path between the 2.



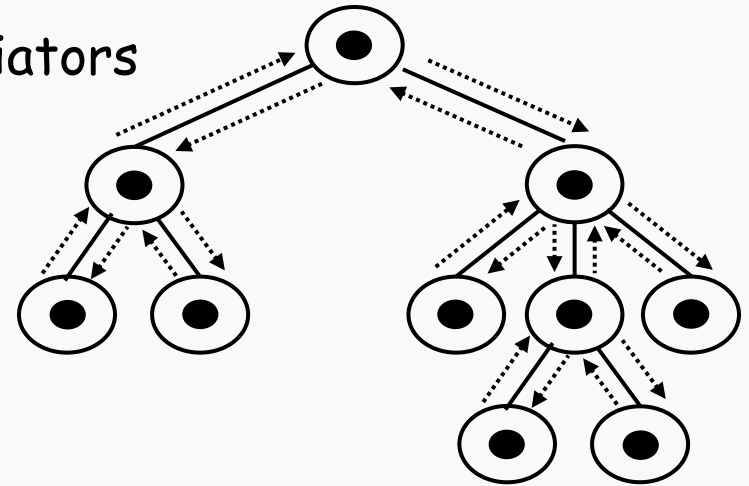
It is not possible that **w** sends **M** both towards
x and **z** !!!

Which nodes become **saturated**
depends on the unpredictable delays

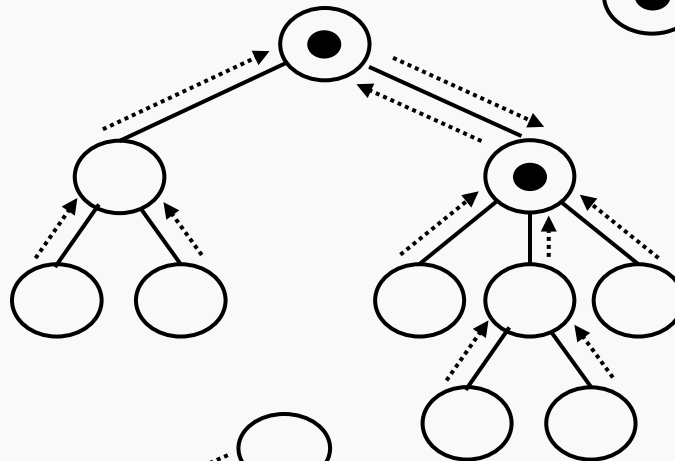
Message Complexity (Tree)

Activation (Awake): Worst case - n initiators

$$2(n-1)$$

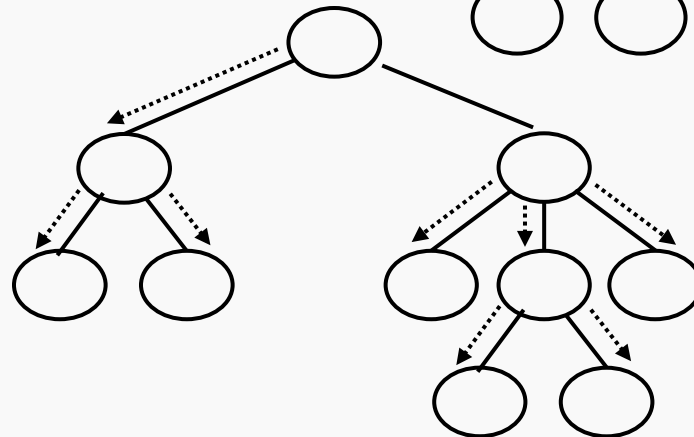


Saturation:



$$n$$

Notification:



$$n - 2$$

Paola Flocchini

$$\text{Tot: } 2n - 2 + n + n - 2 = 4n - 4$$

Message Complexity

Activation: In general - k^* initiators

$$n + k^* - 2 \quad (\text{wake-up in the tree})$$

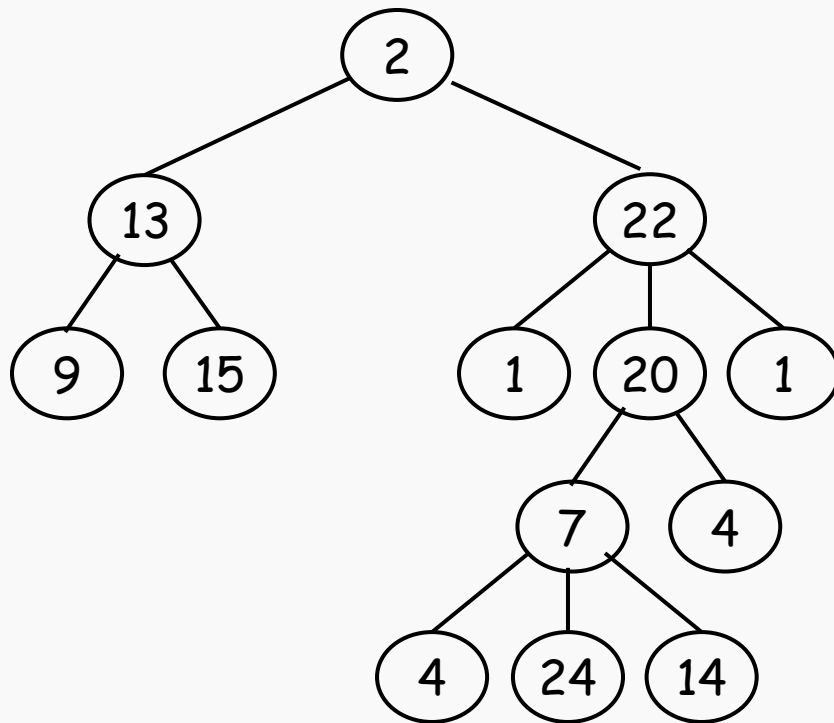
Saturation: n do not depend on number of initiators

Notification: $n - 2$

$$\text{TOT: } 2n + k^* - 2$$

Put information in the saturation message

Minimum Finding



START:

Entity x has in input
 $value(x)$

END:

At the end each entity
should know whether it is
the minimum or not.

Obs. In a *Rooted* Tree **Min-Find** is simple:

exploit strong assumption:

Children --> Parent (**Convergecast**)

Parent --> Children (**Broadcast**)

1. Broadcast (from **Root = Initiator = Leader** to **Leaves**)
2. Convergecast (values from **Leaves** --> **Root**)
3. Broadcast (Notification of the global **Minimum**)

In a simple Tree, we exploit **SATURATION**:

The role of the Root is replaced by the two saturated nodes (**co-leaders**).

Leaves start sending the saturation msg **M** containing their **val(x)**.

when **Internal node** receives all **M-values** from their **children**, they send-up the computed value to their **Parent**

The two **co-leaders** will know the **minimum**. Then they start broadcast **it** to all the tree.

States S {AVAILABLE, ACTIVE, PROCESSING, SATURATED} Sinit = AVAILABLE

AVAILABLE

Spontaneously (Initiators)

send(Activate) to $N(x)$;

min := $v(x)$;

Neighbours := $N(x)$

if $|Neighbours|=1$ then

M := ("Saturation", min); (I am a Leaf)

parent $\leftarrow$ Neighbours;

send(M) to parent;

become PROCESSING;

else become ACTIVE;

```
Receiving(Activate )  
  send(Activate) to N(x) - {sender};  
  min:=v(x);  
  Neighbours:= N(x);  
  if |Neighbours|=1 then  
    M:=("Saturation", min); I am a LEAF  
    parent ← Neighbours;  
    send(M) to parent; Start Saturation Phase  
    become PROCESSING;  
  else become ACTIVE;
```

ACTIVE

Receiving(M)

min := MIN{min, M}

Neighbours := Neighbours - {sender};

if |Neighbours|=1 then

M := ("Saturation", min);

parent $\leftarrow$ Neighbours;

send(M) to parent;

become PROCESSING;

Bottom-Up : Saturation towards the co-leaders

PROCESSING

receiving(M)

min := MIN{min, M}

I am a co-Leader

Notification := ("Resolution", min)

send (Notification) to N(x) - parent

if v(x)=min then

become MINIMUM

else

become LARGE

receiving(Notification)

send(Notification) to N(x) -parent

if v(x)=Received_Value then

become MINIMUM; Broadcast and Update

else

become LARGE;

Paola Flocchini

Distributed Function Evaluation.

Saturation can be used to compute a general class of functions in a distributed tree.

Which ones?

Which are the crucial properties of such functions?

F must be a **semi-group** operation

- associative $F(a,b,c) = F(F(a,b),c)$

-commutative $F(a,b,c) = F(c,a,b)$

In this case,
the saturation technique works!!!

Minimum, Maximum, Sum, Product,
Logical Predicates

Just change msg M and other small tuning

What about Computing

the **Average** of the node **values** ?

Obs. Average is **not** a semigroup operation:

Avg(4,4,4,4, 45) < < < **Avg**(**Avg**(4,4,4,4), 45)

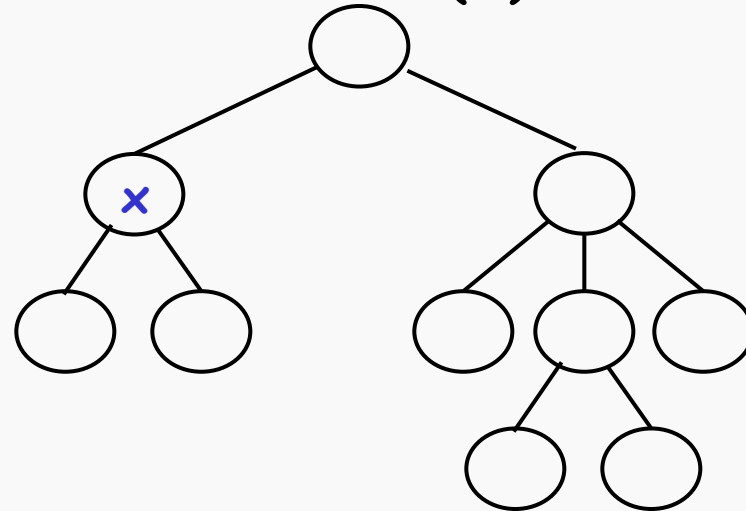
Exercise: find an **optimal** Protocol for
Avg on a Tree.

Hint--: Don't look at the book!!!!

Finding Eccentricities

$d(x,y)$ = distance between x and y

$\text{Max}_{y \in V} \{d(x,y)\} = r(x)$ eccentricity of x $r(x) = 4$



How to find the eccentricity of **all** nodes?

Idea 1:

- 1) EVERY NODE **BROADCASTS** A REQUEST,
- 2) THE LEAVES SEND BACK A MESSAGE TO COLLECT THE DISTANCES (**CONVERGECAST**).

Message Complexity: $O(n \cdot n = n^2)$

There is a lot of **redundant** computations !!

Can we **recycle** some computed values??!

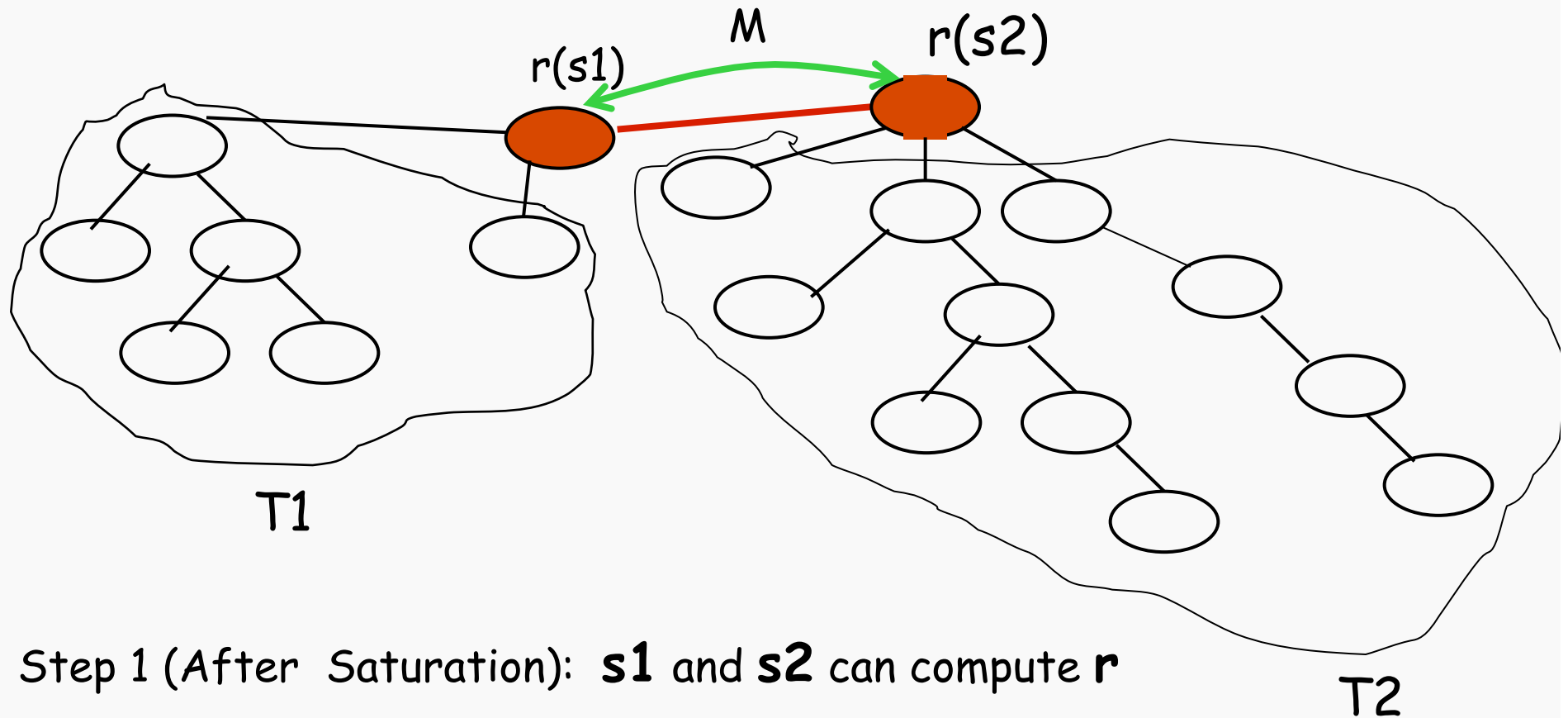
Better Idea:

Again use the Saturation Technique:

- 1) FIND THE ECCENTRICITY OF THE
TWO **SATURATED** NODES BY USING MSG M (SATUR. PHASE)
- 2) PROPAGATE THE **NEEDED** INFO SO THAT THE
OTHER NODES CAN FIND THEIR ECCENTRICITY
(IN THE NOTIFICATION PHASE)

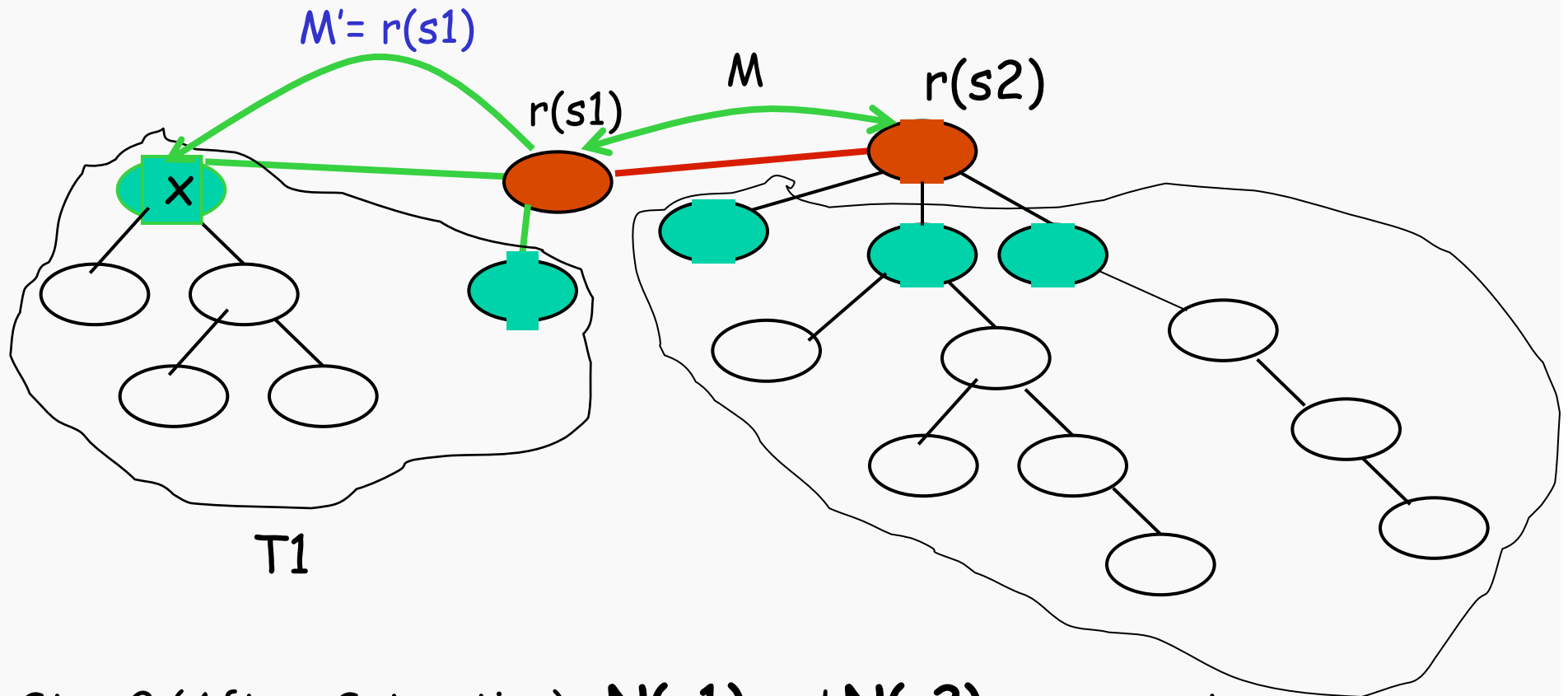
Complexity = saturation

$$\text{THM. } r(s1) = \max\{h(T1), 1 + r(s2)\}$$



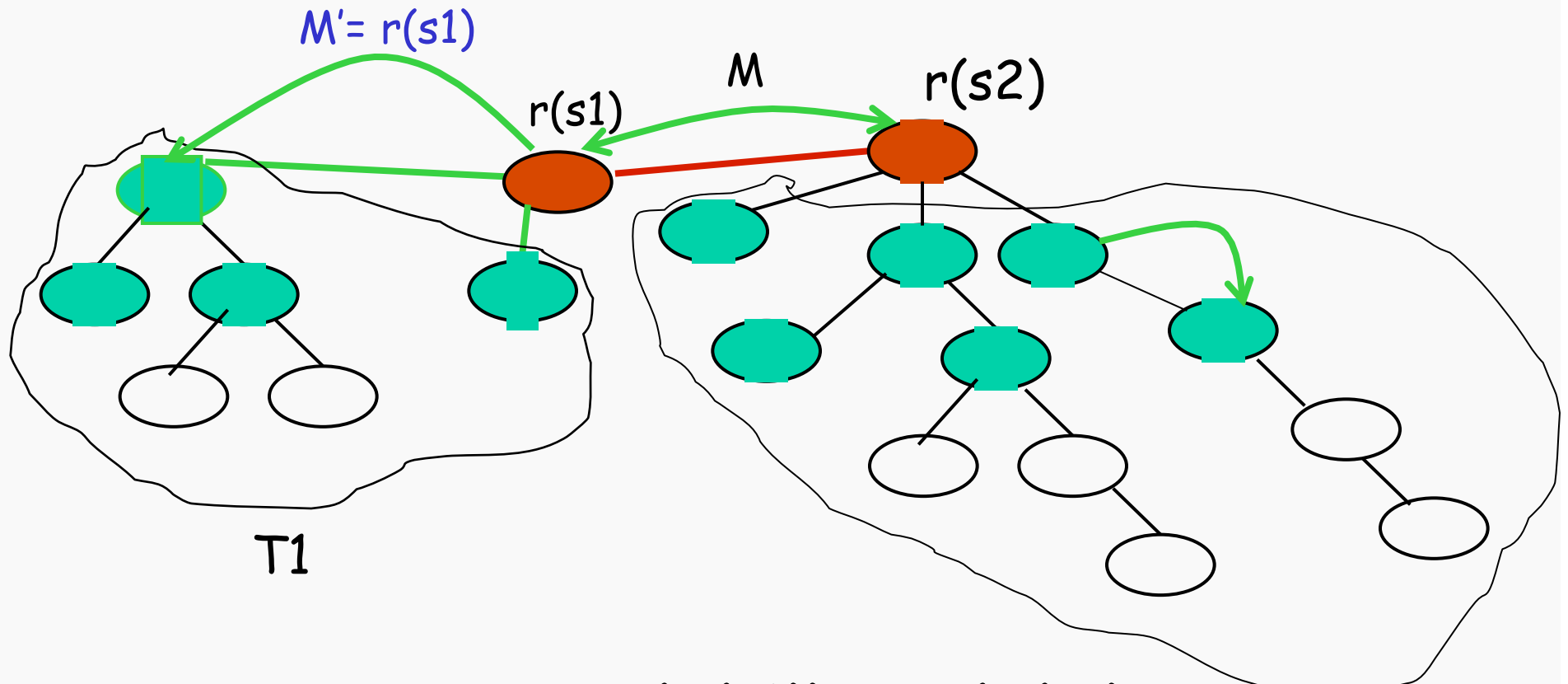
Step 1 (After Saturation): **s1** and **s2** can compute **r**

$$\text{THM. } r(x) = F(\text{knw}(x) , M' + 1)$$



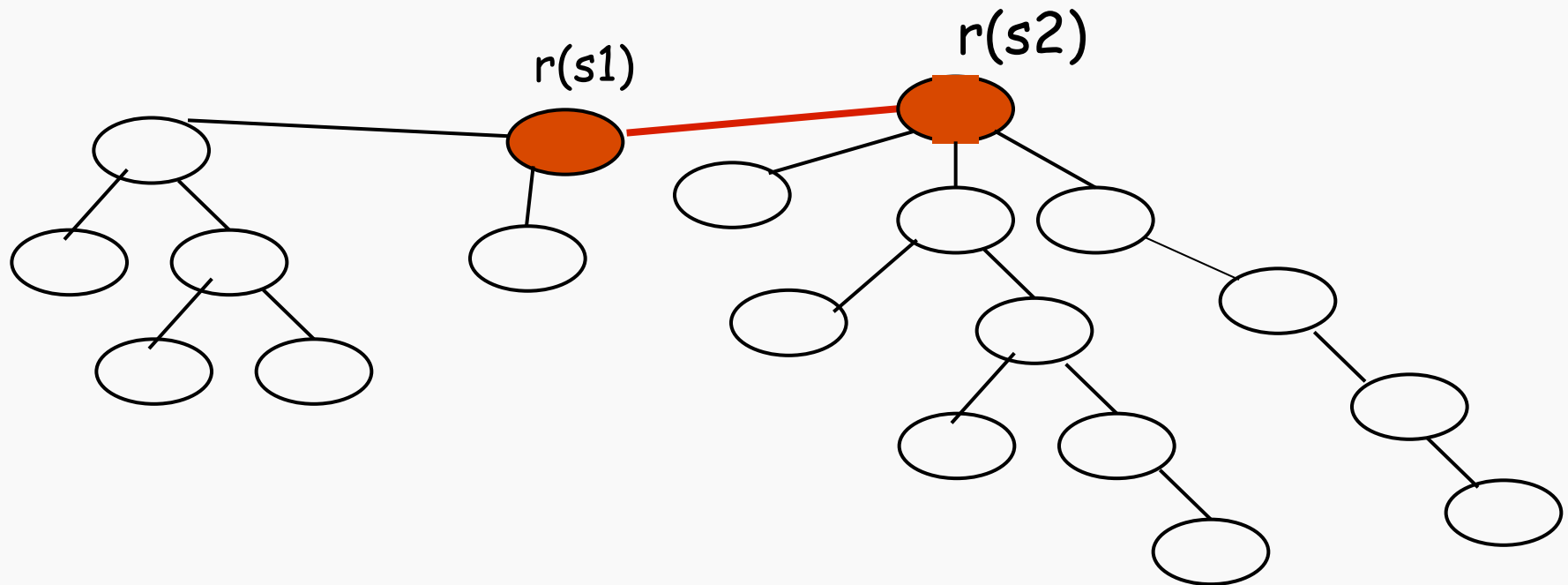
Step 2 (After Saturation): $N(s1)$ and $N(s2)$ can compute r

THM. and so on....



Step 3 (After Saturation): $N(N(s_1))$ and $N(N(s_2))$ can compute r

The Protocol



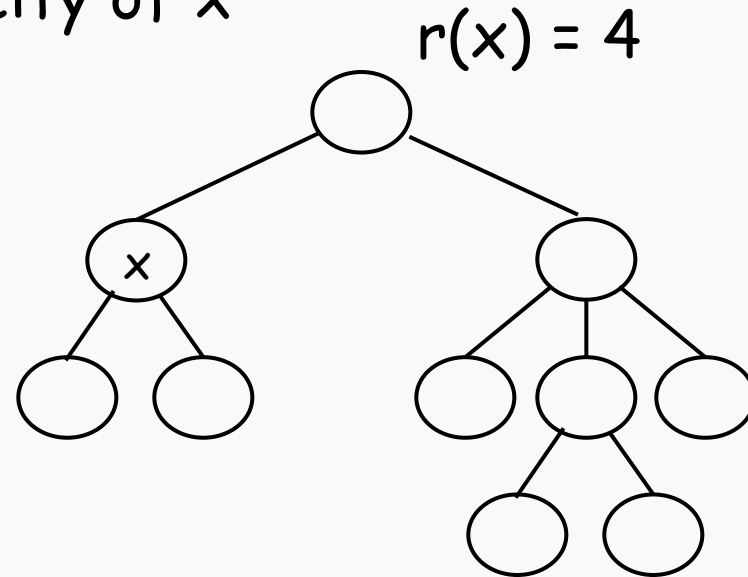
States $S \{AVAILABLE, ACTIVE, PROCESSING,$
 $SATURATED\}$ $S_{init} = AVAILABLE$

Paola Flocchini

Finding Eccentricities

$d(x,y)$ = distance between x and y

$\text{Max}_{y \in V} \{d(x,y)\} = r(x)$ eccentricity of x



How to find the eccentricity of all nodes

Idea 1:

- 1) EVERY NODE BROADCASTS A REQUEST,
- 2) THE LEAVES SEND UP A MESSAGE TO COLLECT THE DISTANCES.

Complexity: $O(n^2)$

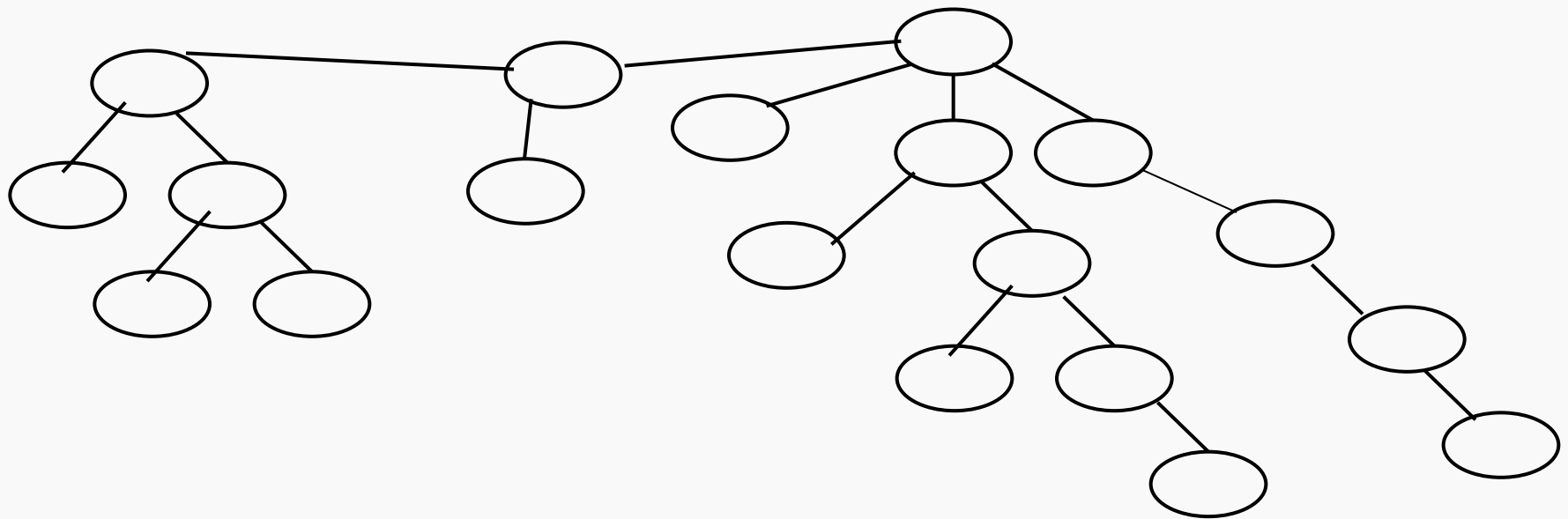
Other Idea:

Based on the saturation technique:

- 1) FIND THE ECCENTRICITY OF THE TWO SATURATED NODES
- 2) PROPAGATE THE NEEDED INFO SO THAT THE OTHER NODES CAN FIND THEIR ECCENTRICITY (IN THE NOTIFICATION PHASE)

Complexity = saturation

Observations and Examples



States $S \{AVAILABLE, ACTIVE, PROCESSING, SATURATED\}$ $S_{init} = AVAILABLE$

Paola Flocchini

Idea.

- Define a local variable **maxdist(y)** that gets the **Depth** of the *Subtree* rooted on **y**.
- When computed, **y** sends **maxdist(y)** to its parent
- When a PROCESSING node **x** receives from its Parent (so **x** becomes saturated)

$$M = (\text{Saturation}, \text{maxdist})$$

Then:

Then,

1. x can compute its **eccentricity**
2. x can send the **correct information** to each of its sons:

x sends to y

$$\text{MAX}\{ \text{maxdist}(z) \mid z \# y \text{ is a son of } x \}$$

define Distance[]

AVAILABLE

Spontaneously

send(Activate) to N(x);

Distance[x] := 0;

Neighbours := N(x)

if |Neighbours| = 1 then

maxdist := 1 + Max{Distance[*]}

M := ("Saturation", maxdist);

parent $\leftarrow$ Neighbours;

send(M) to parent;

become PROCESSING;

else become ACTIVE;

Receiving(Activate)

send(Activate)to $N(x) - \{\text{sender}\};$

Distance $[x] := 0;$

Neighbours $:= N(x);$

if $|Neighbours|=1$ **then**

maxdist $:= 1 + \text{Max}\{\text{Distance}[*]\}$

M $:= ("Saturation", \text{maxdist});$

parent $\leftarrow Neighbours;$

send(M) **to** **parent**;

become **PROCESSING**;

else **become** **ACTIVE**;

ACTIVE

Receiving(M)

Distance[{sender}] := Received_distance;

Neighbours := Neighbours - {sender};

if |Neighbours|=1 then

maxdist := 1 + Max{Distance[*]}

M := ("Saturation", maxdist);

parent $\leftarrow$ Neighbours;

send(M) to parent;

become PROCESSING;

PROCESSING

receiving(M)

Distance[{ sender}]:= Received_distance;

$r(x) := \text{Max} \{ \text{Distance}[z] : z \in N(x) \}$

for all $y \in N(x) - \{\text{parent}\}$ **do**

 maxdist := 1 + Max{Distance[z]:
 $z \in N(x) - \{y\}$

send("Resolution", maxdist) **to** y

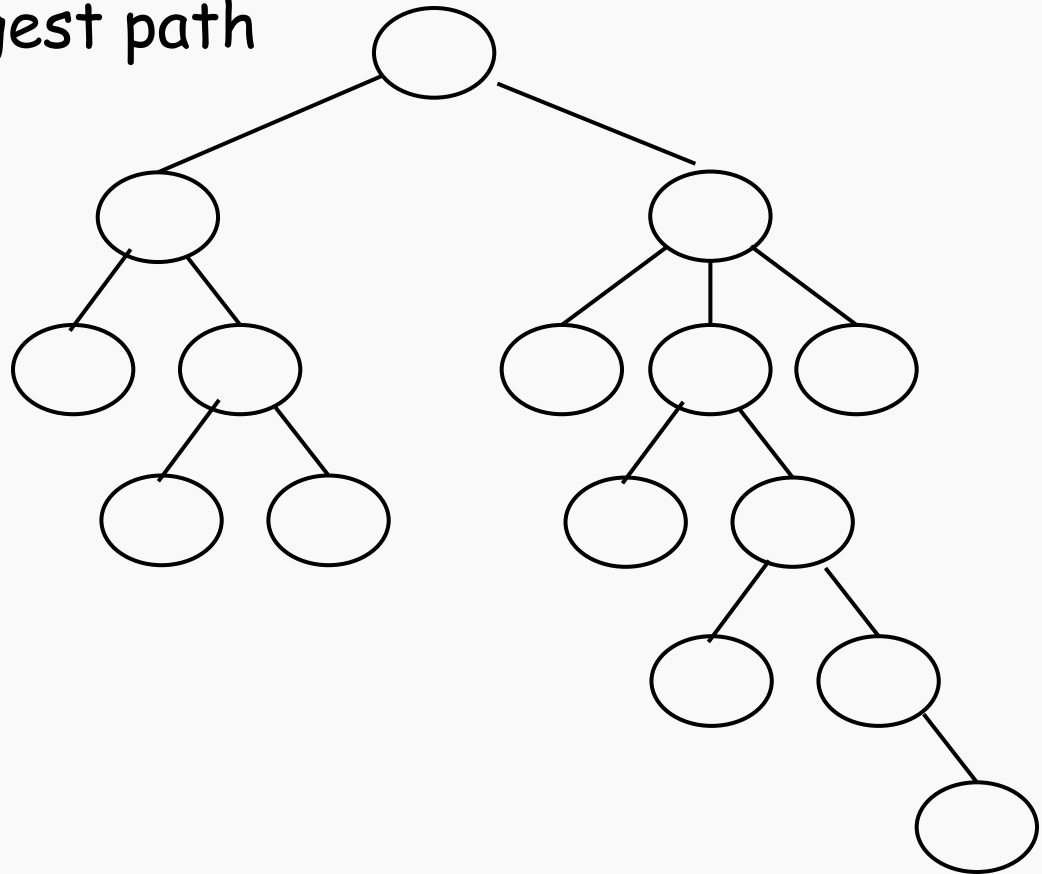
endfor

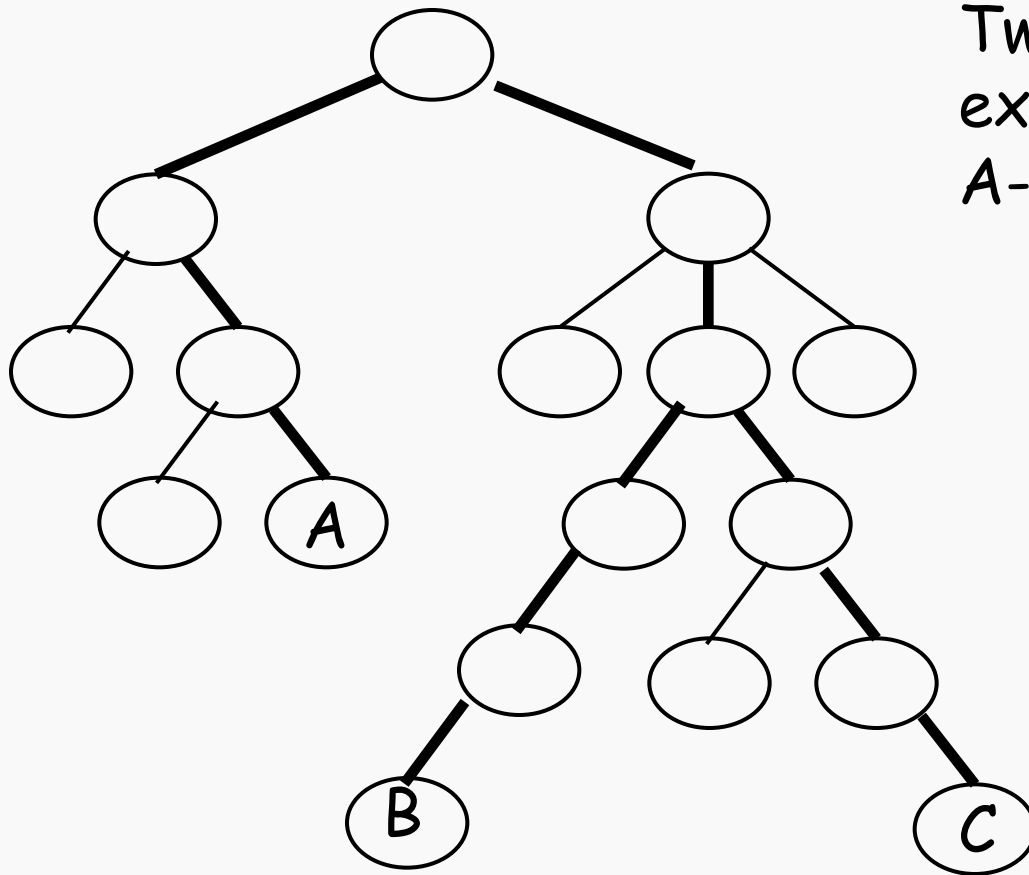
become DONE

Center Finding

c is the center if $r(c) \leq r(x)$ for all x belonging to V . Max distance is minimized.

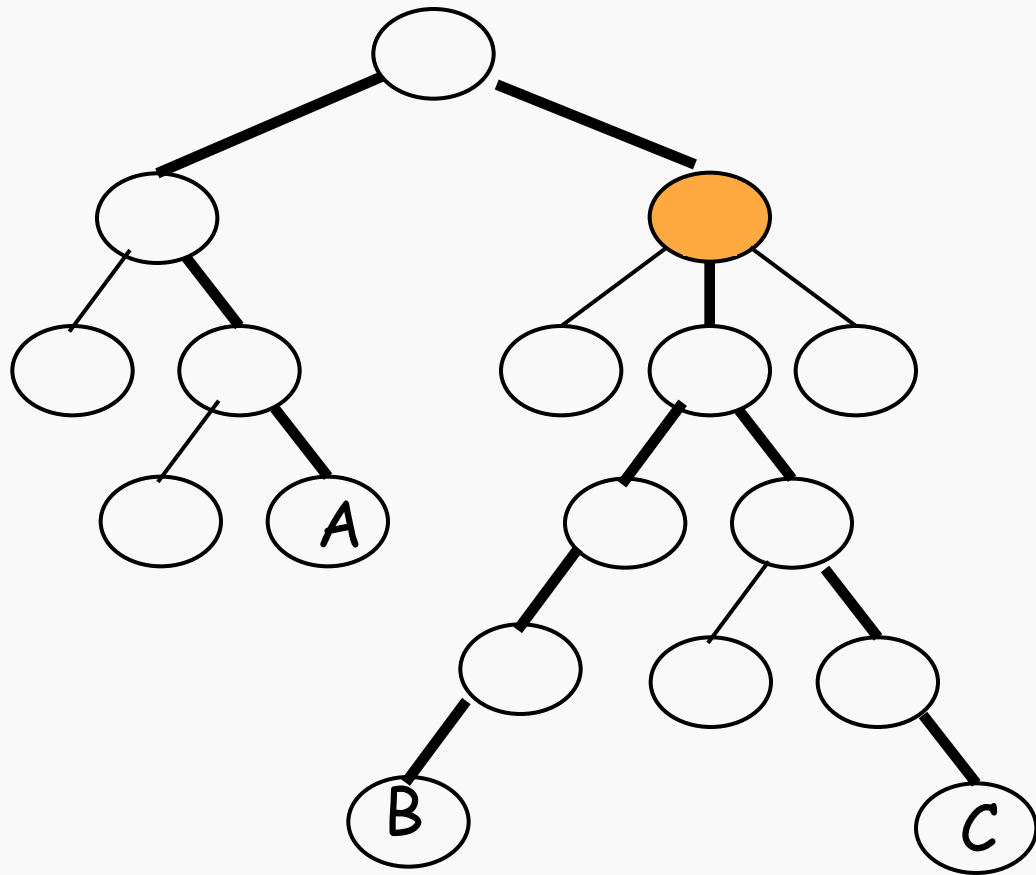
Diametral path: Longest path





Two diameters in this
example:
A-B, and A-C

The center is the
node with smallest
eccentricity



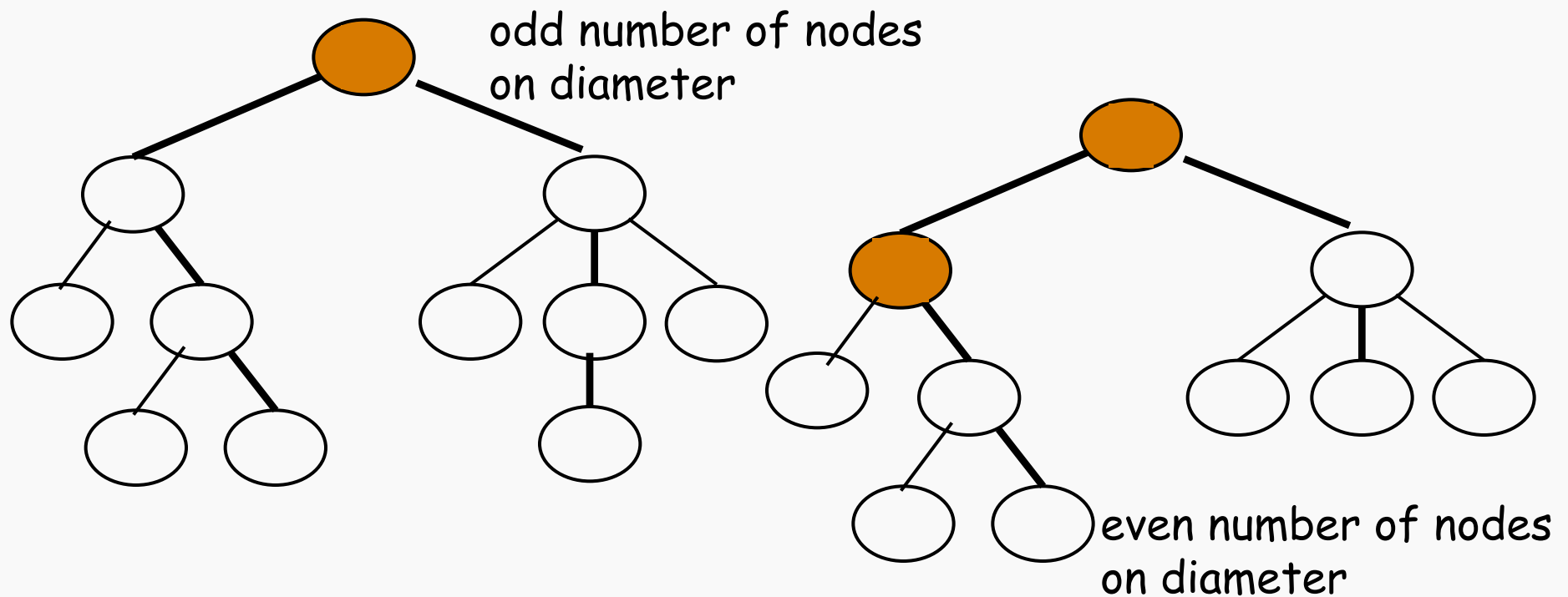
One Idea:

- 1) FIND ALL THE ECCENTRICITIES
- 2) FIND THE SMALLEST

$$\begin{matrix} 4n-4 \\ 2n-2 \end{matrix} \rightarrow 6n-6$$

A better strategy can be devised exploiting properties of the center.

Property 1: In a tree there is a unique center, or there are two centers that are neighbours



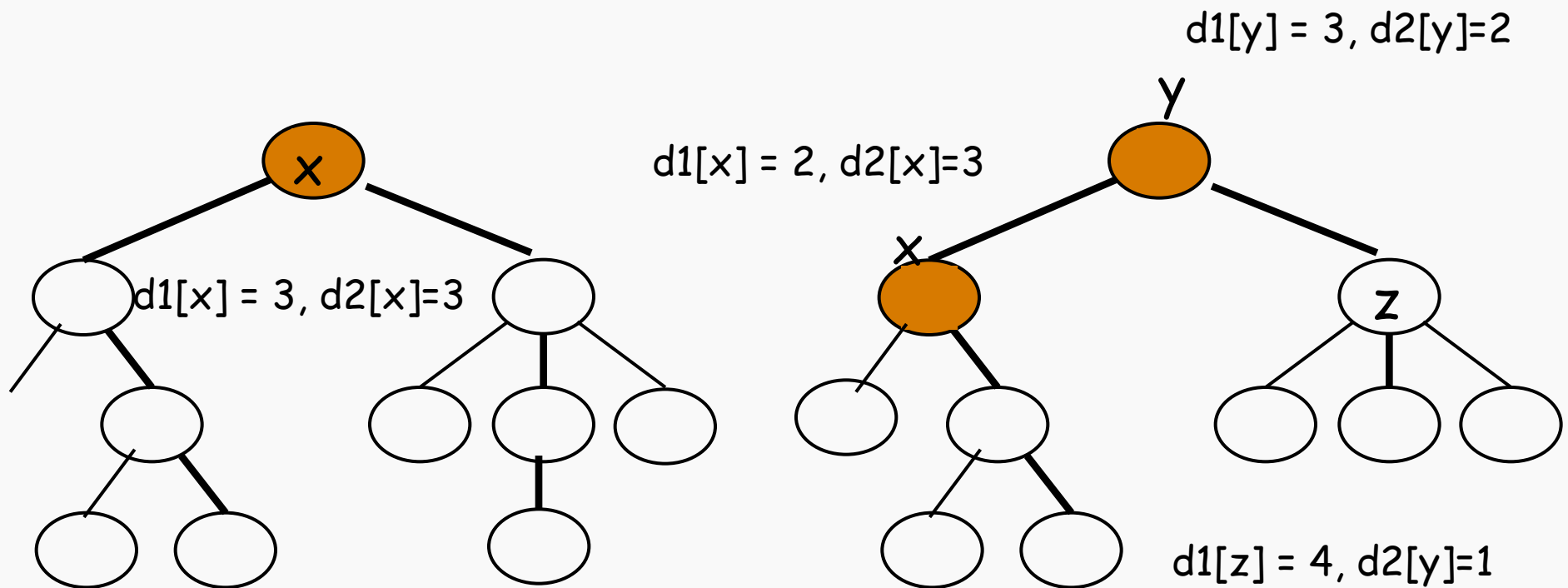
Proposition 2: Centers are on diametral paths.

$d1[x] = \text{max dist}$ $d2[x] = \text{second max dist (through different neighbours)}$

Property 3: A node x is a center iff

$$d1[x] - d2[x] \leq 1$$

(if $d1[x] = d2[x]$ there is only one center).



Another Idea:

- 1) FIND ALL THE ECCENTRICITIES
- 2) EACH NODE CAN FIND OUT LOCALLY WHETHER
IT IS THE CENTER OR NOT

$4n-4$

Yet another better idea:

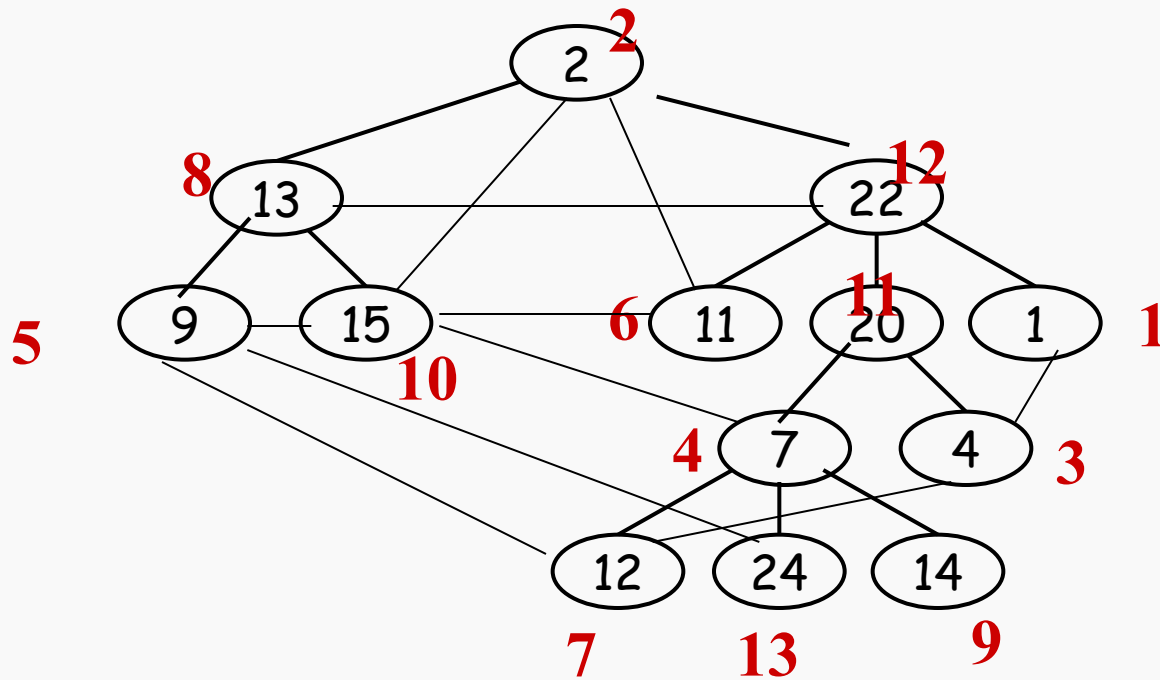
- 1) FIND THE ECCENTRICITIES OF THE SATURATED NODES $3n-2$
- 2) LOCALLY CHECK IF I AM THE CENTER (CHECKING LARGEST AND SECOND LARGEST)
- 3) IF I AM NOT THE CENTER, PROPAGATE THE DISTANCE INFORMATION ONLY IN THE DIRECTION OF THE CENTER $\leq n/2$

How do I know the direction of the center ?

Examples
Paola Flocchini

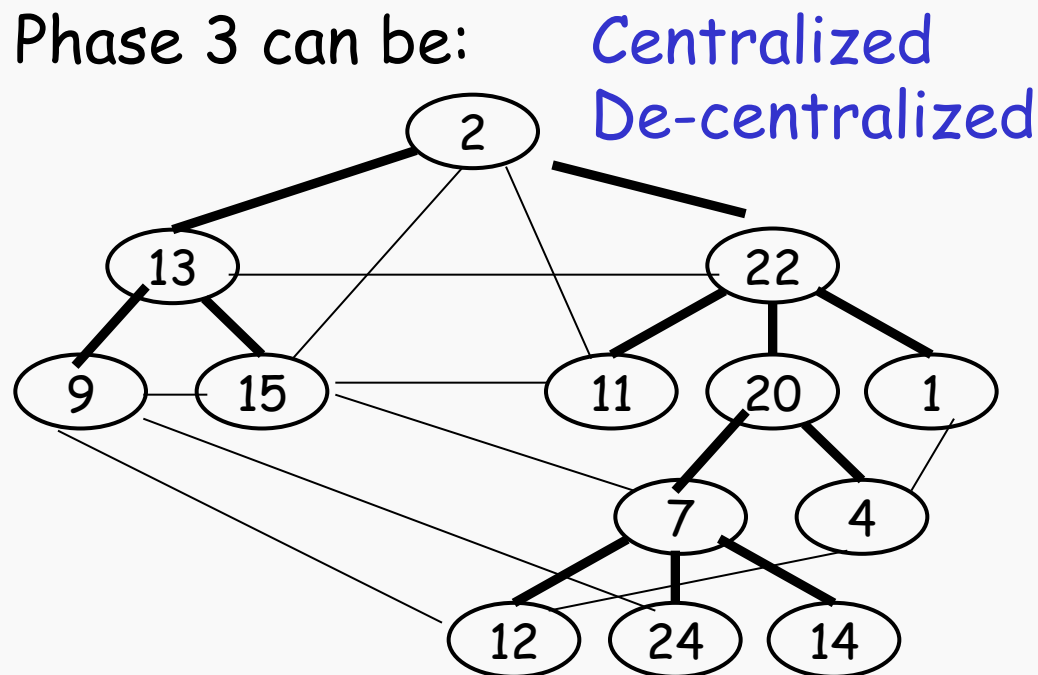
$\leq 3.5 n-2$

Ranking

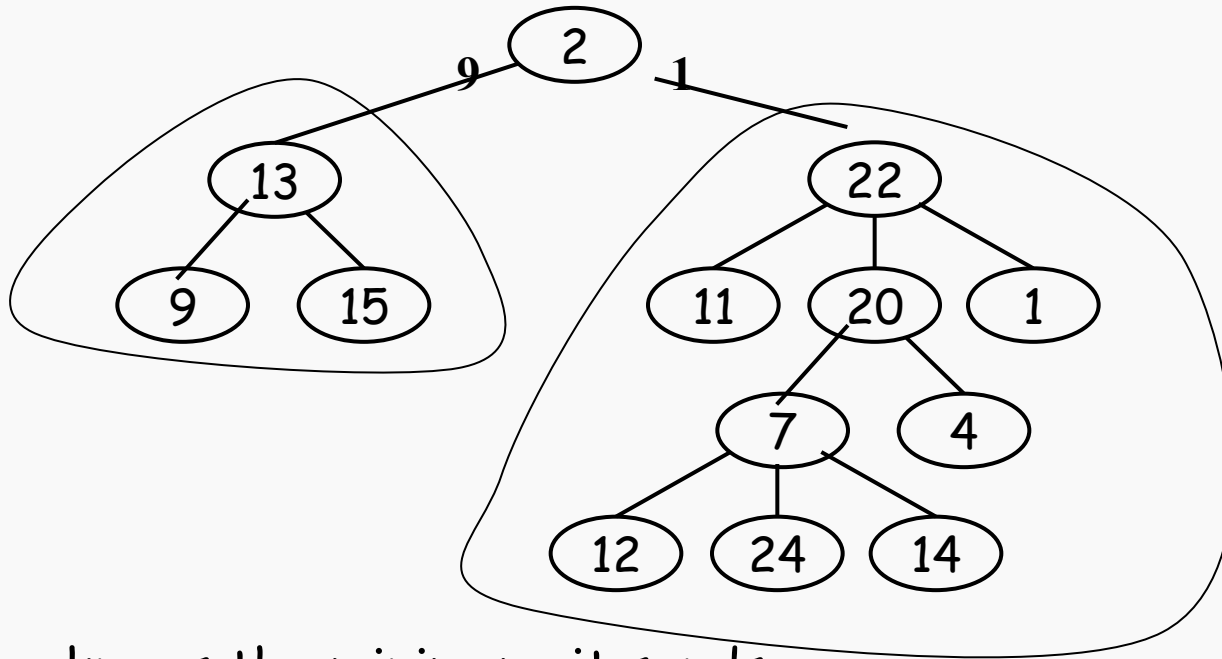


In an arbitrary network:

- 1) Find a spanning tree
- 2) Use saturation+ minimum finding to find a starting node
- 3) Do-ranking



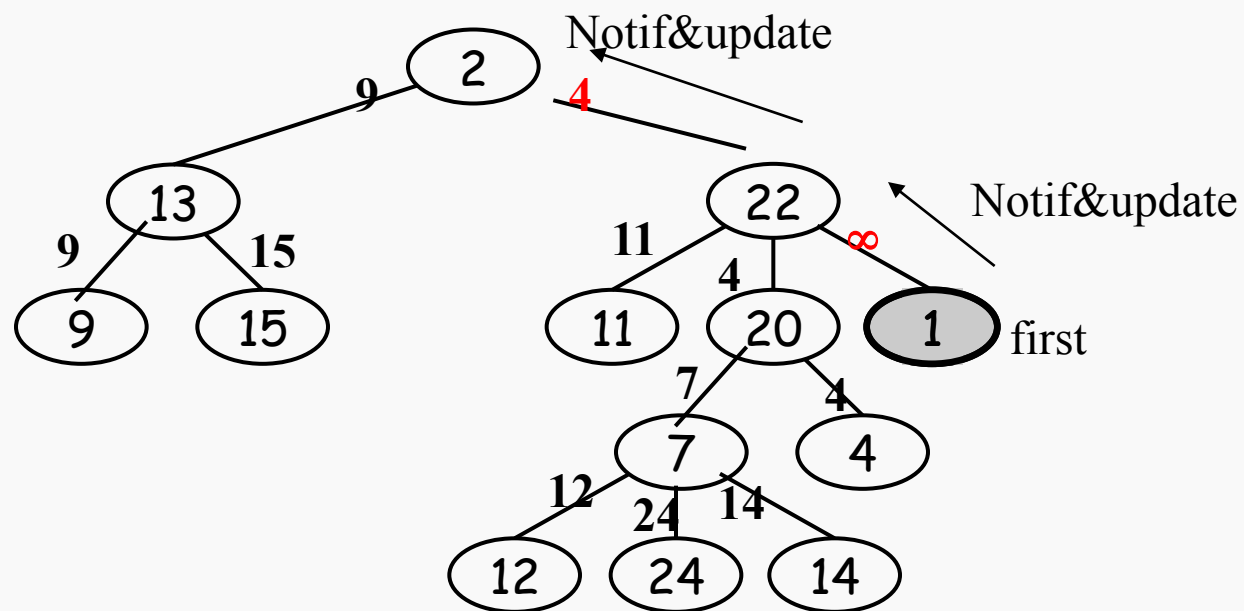
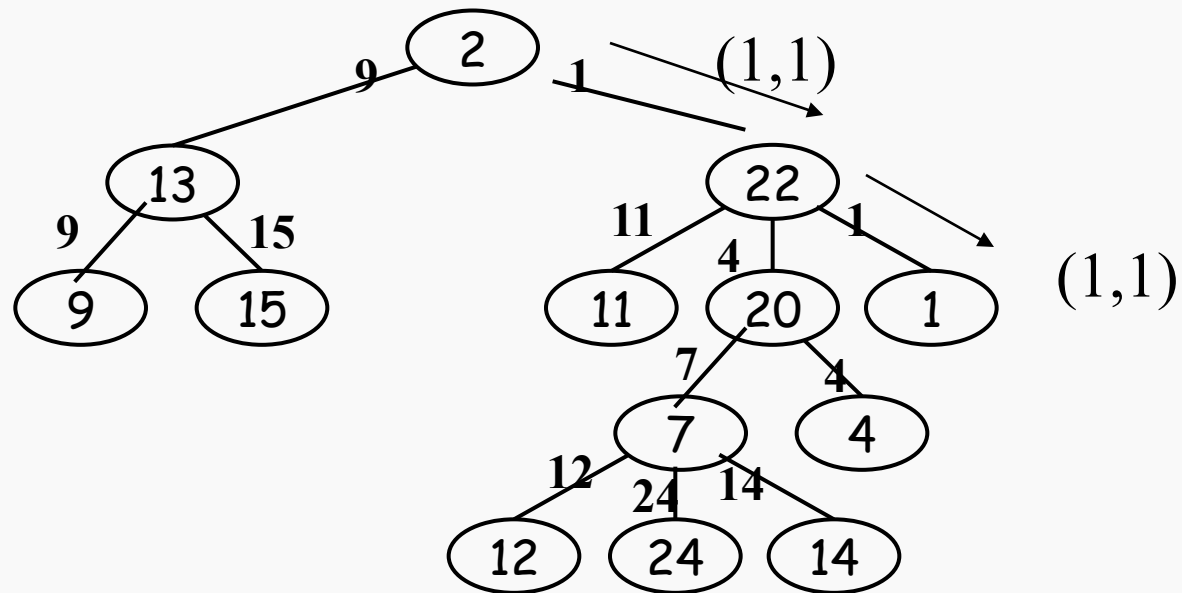
Centralized Ranking

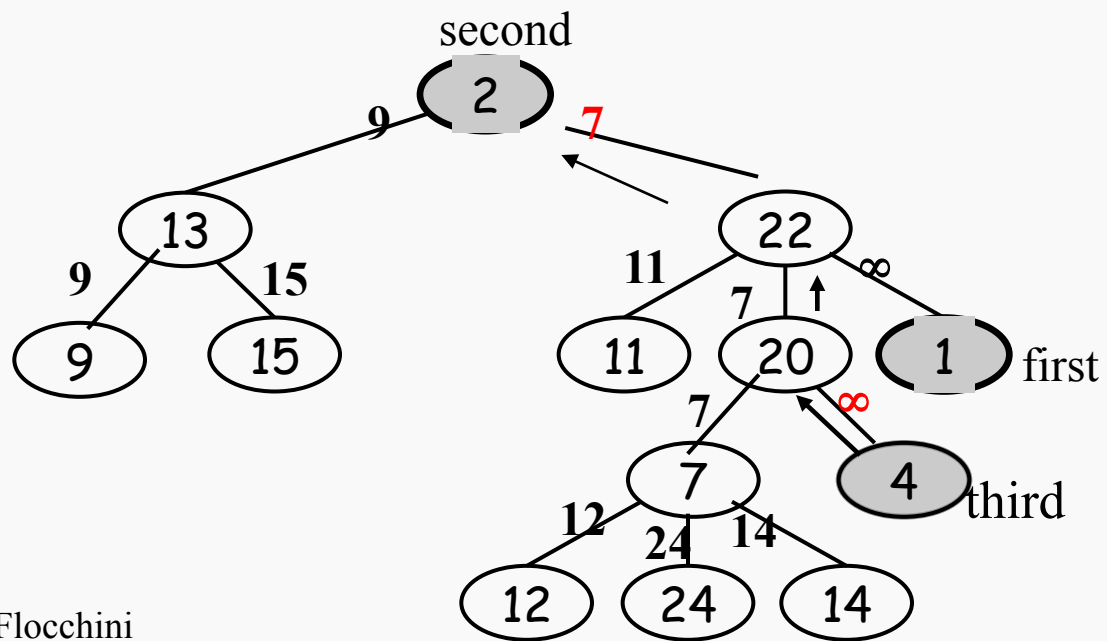
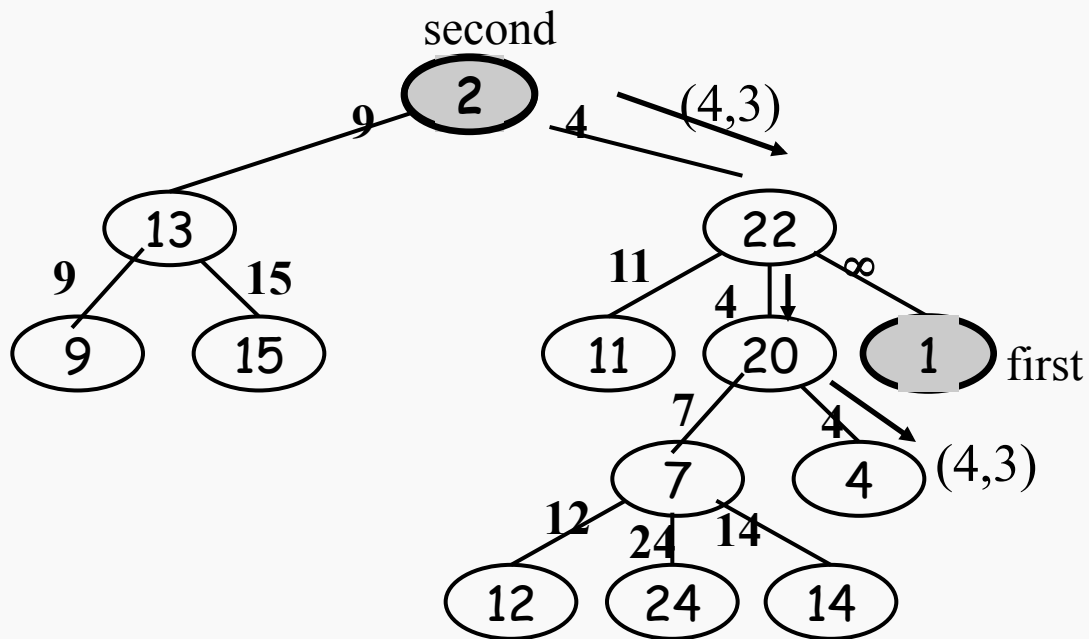


The leader knows the minimum, it sends in that direction a ranking message

Every node knows the minimum in its subtrees, they can then forward the ranking message in the right direction

When the node to be ranked receives the message
It sends up a notification&update message that will travel up to the leader

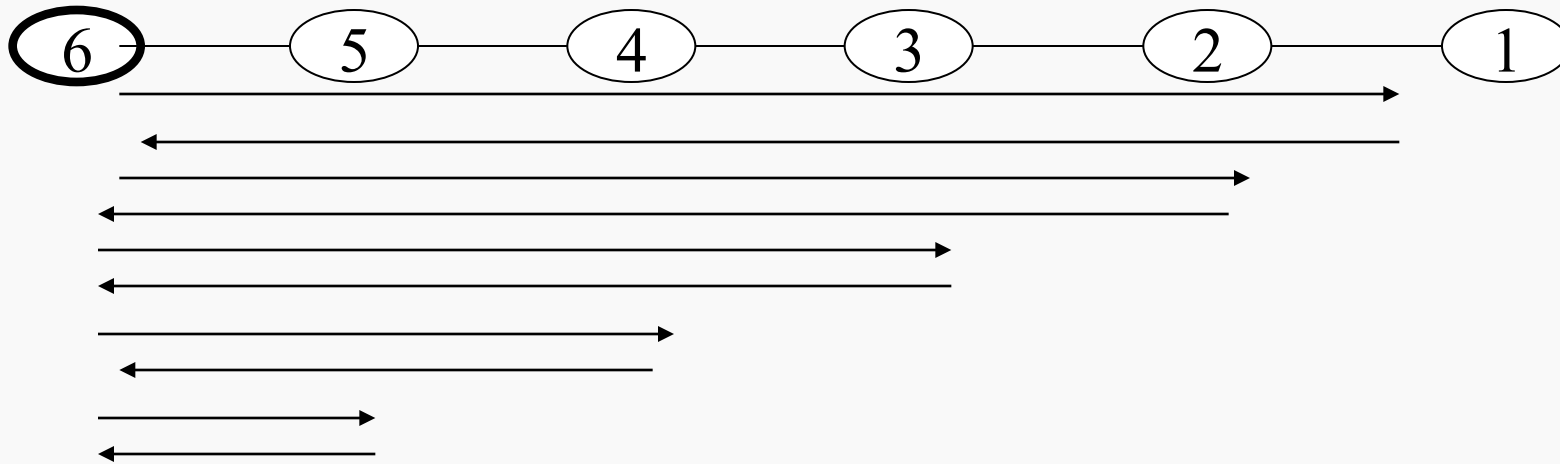




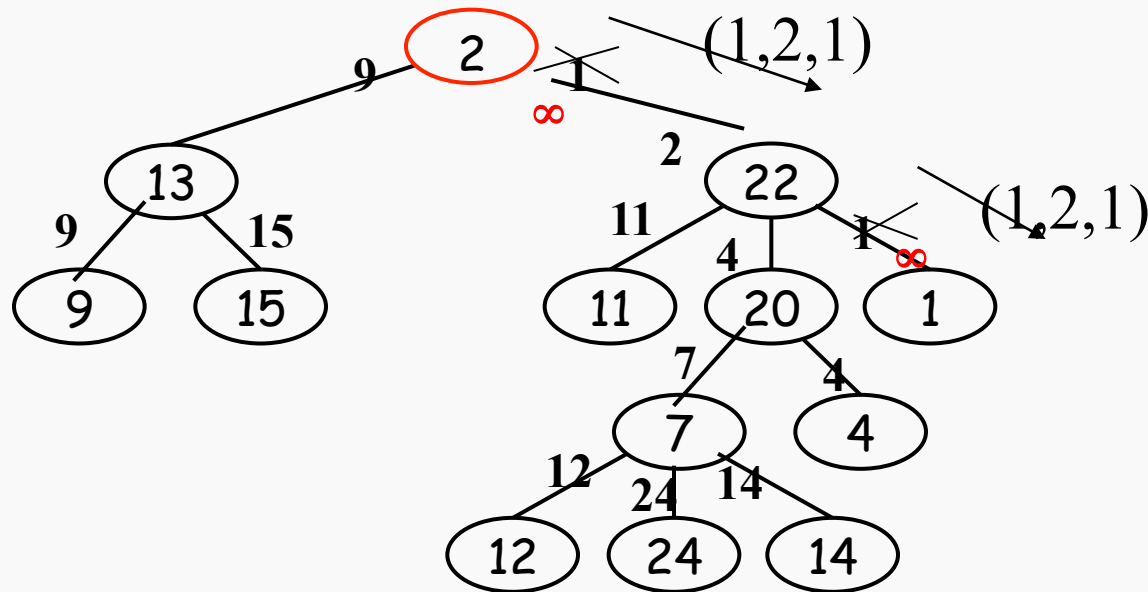
Paola Flocchini

Etc...

Complexity: worst case



Decentralized Ranking



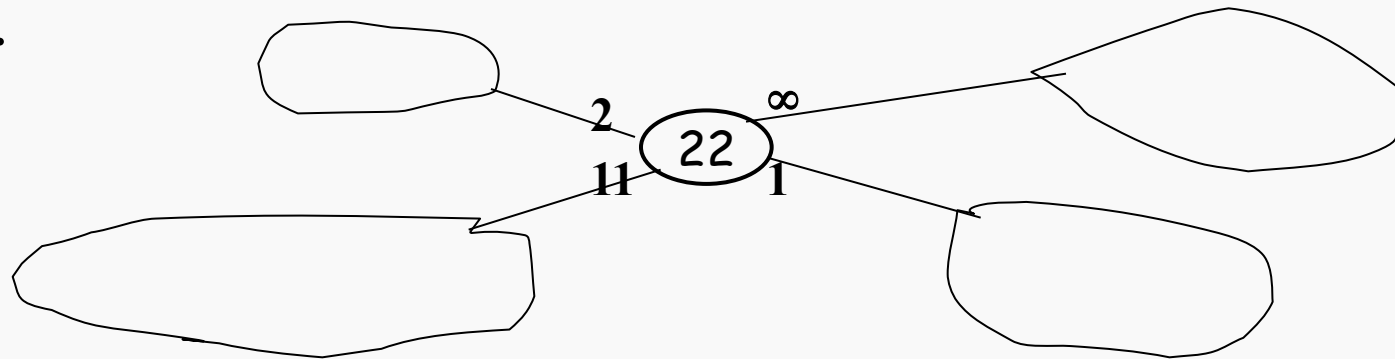
The starter node send a ranking message of the form: **(first, second,rank)** in the direction of first.

first: smallest value

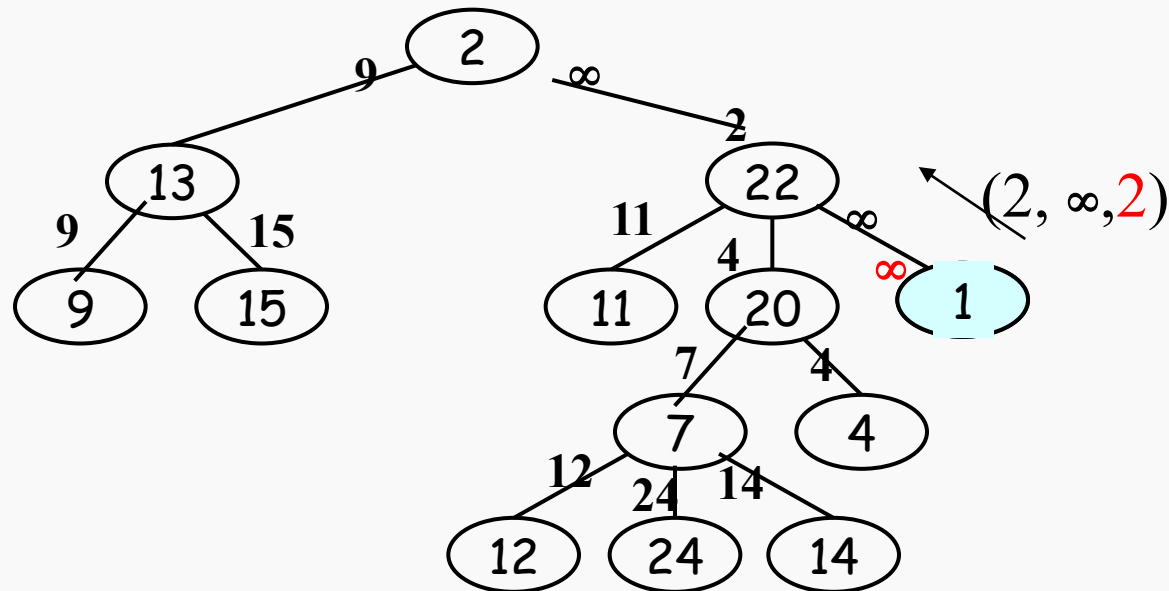
second: second smallest known **SO FAR**

(this is a guess on the value that has to be ranked after first)

The value on a link indicates the
SMALLEST value in the corresponding
subtree.



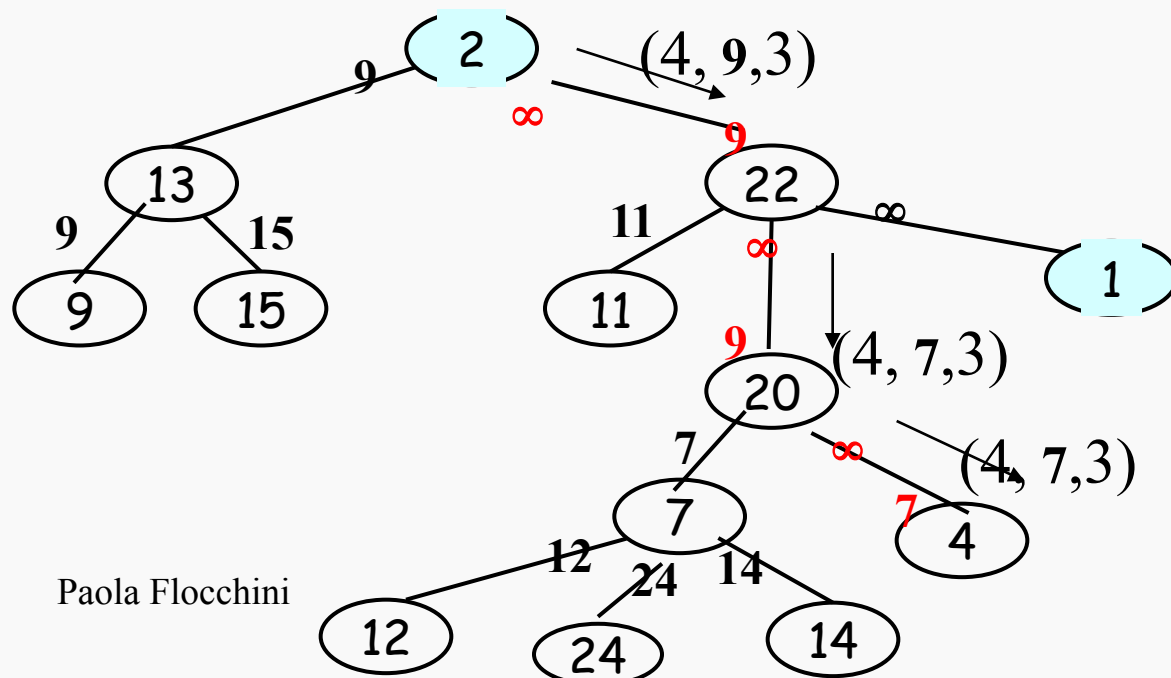
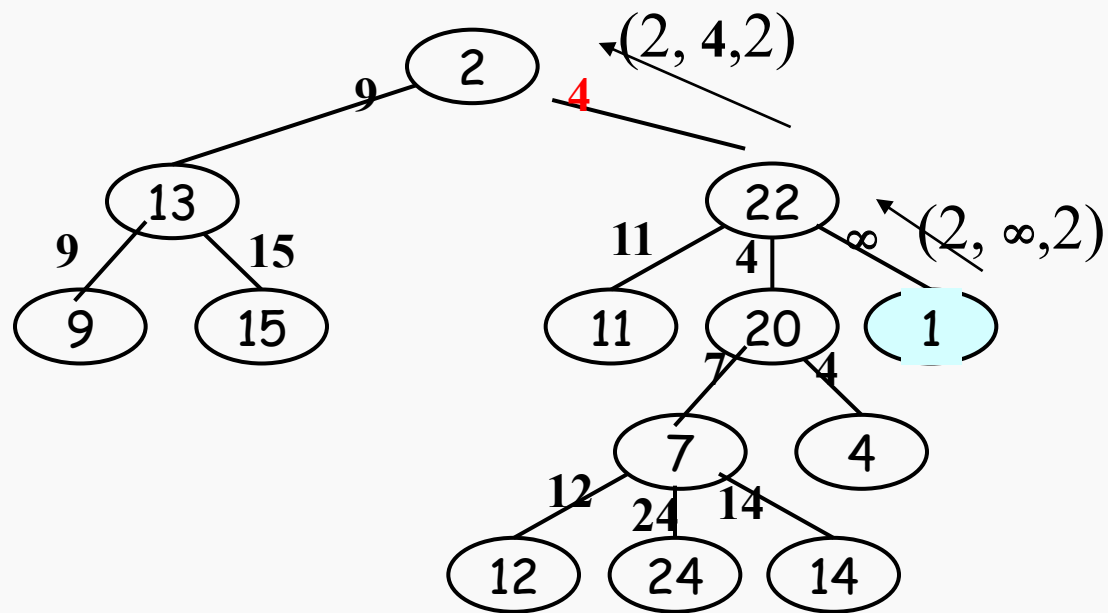
If no value is indicated (or the value is ∞)
it means that the smallest in the
corresponding subtree is unknown
(for the moment)



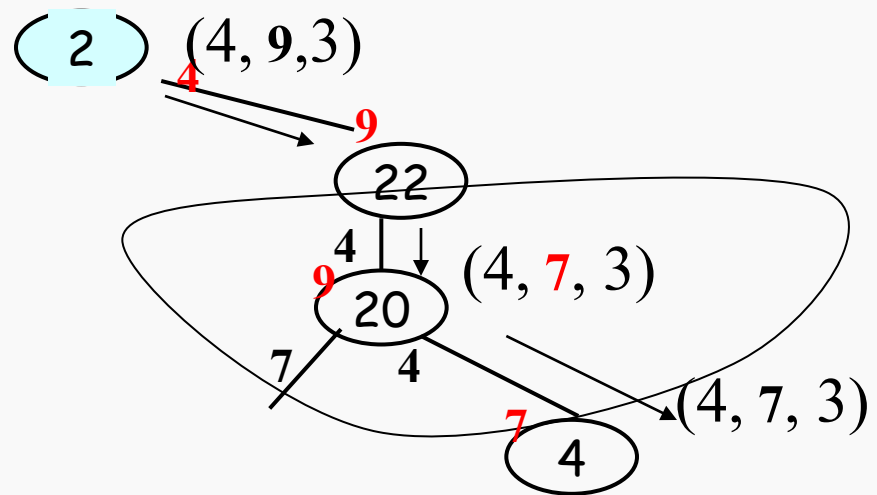
The ranked node attempts to send a ranking message to the next node to be ranked

Second might now be unknown, in this case the value ∞ is used

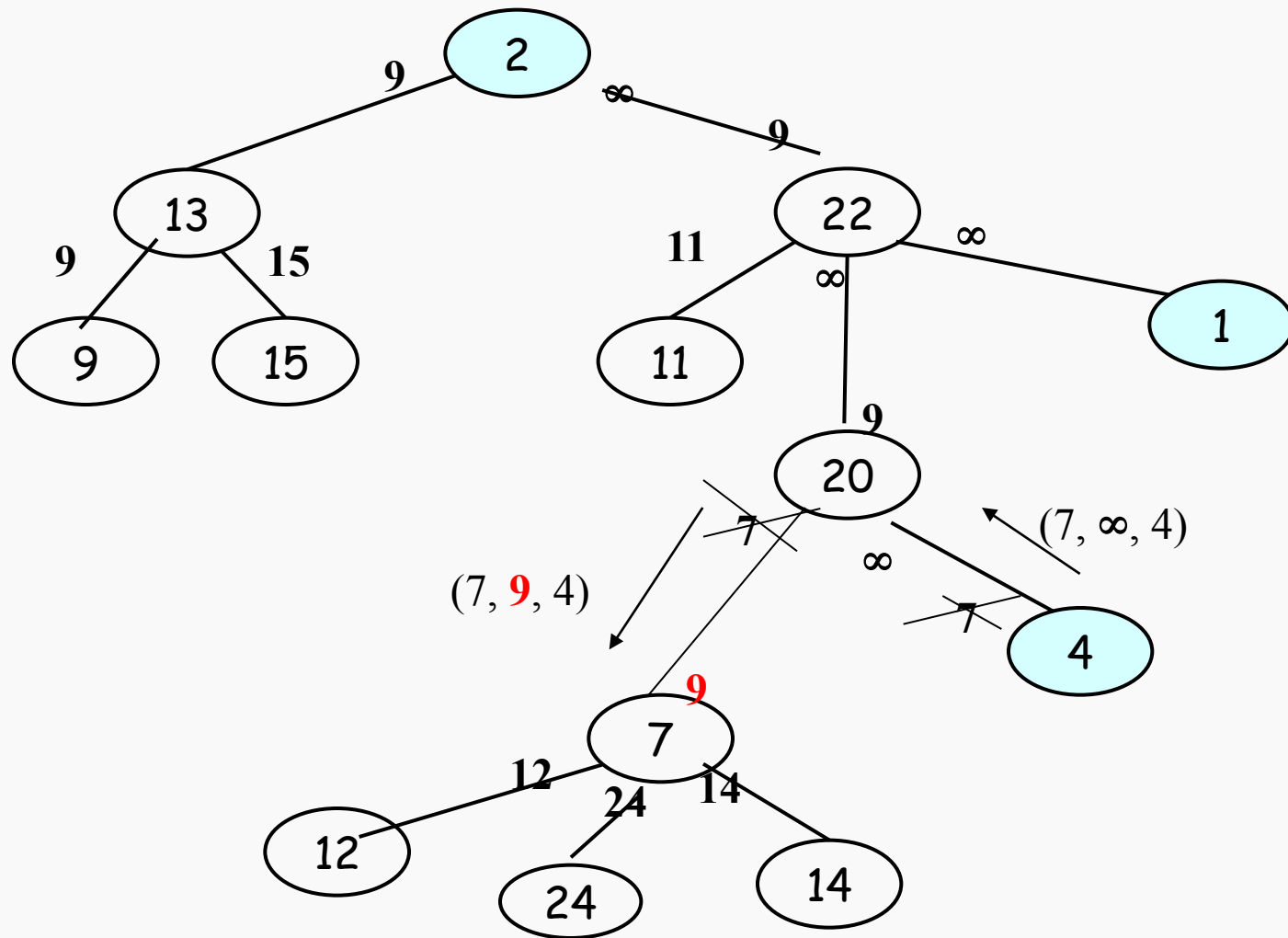
The second variable of the rank message is updated during its travel and the minimum values on the links of the tree are also updated

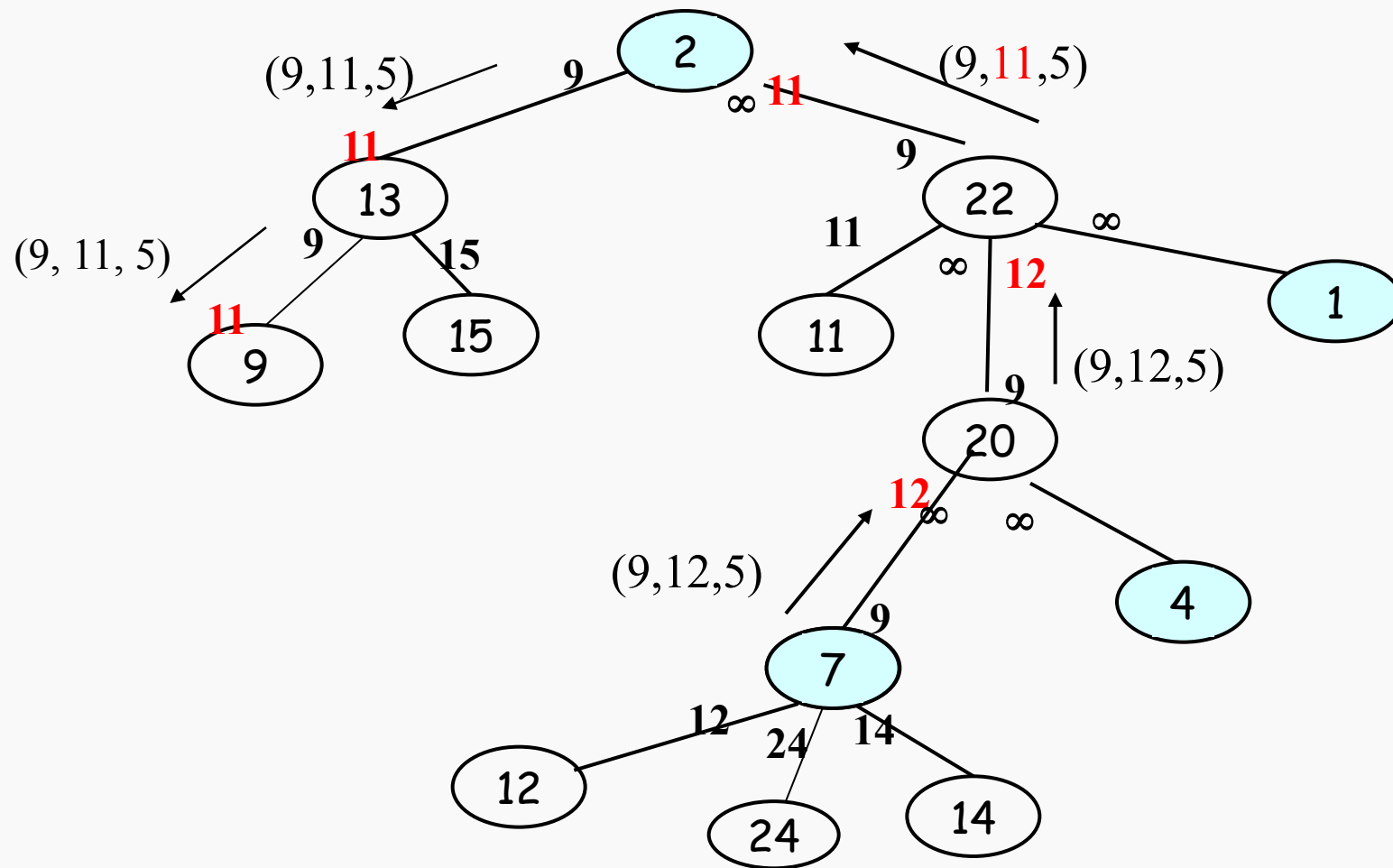


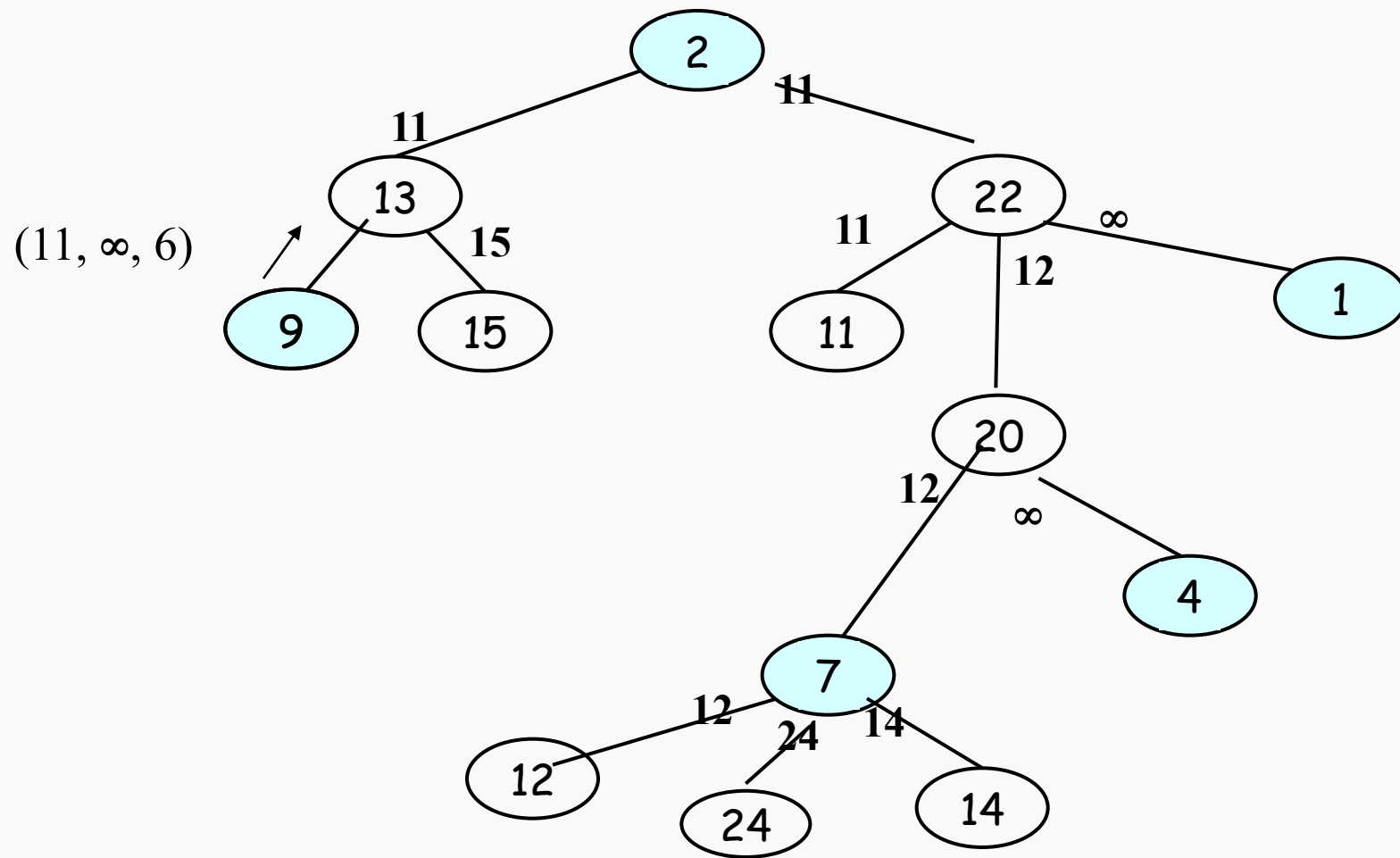
Paola Flocchini



Notice the update







Complexity: worst case

